

09:00-09:50 (쉬는 시간 11:30-12:00)

오디오는 먼저 파일 계약부터 확인합니다

길이·채널·샘플링레이트를 읽고 전사 가능 여부를 판단한다.

진행

강의 12분 · 강사 시연 10분 · 소프트웨어
실습 23분 · 실행 확인 5분

사용 파일

data/demo_meeting.wav
data/demo_meeting_transcript.txt

확인할 결과

audio_metadata.json

2일차는 여덟 개 차시로 하나의 서비스를 완성합니다

오전 3개 차시 뒤 30분 휴식, 12:00-13:00 점심, 오후 5개 차시 뒤 휴식과 Q&A로 마무리합니다.

시간	차시	배우는 내용	진행	확인할 결과
09:00-09:50	1차시	오디오는 먼저 파일 계약부터 확인합니다	강의 12 시연 10 소프트웨어 실습 23 확인 5분	audio_metadata.json
09:50-10:40	2차시	faster-whisper는 로컬 STT의 교체 가능한 adapter입니다	강의 12 시연 10 소프트웨어 실습 23 확인 5분	transcript.json
10:40-11:30	3차시	전사 성공과 전사 품질 통과는 다른 상태입니다	강의 12 시연 10 소프트웨어 실습 23 확인 5분	quality_gate.json
13:00-13:50	4차시	회의록 schema는 요약보다 먼저 합의합니다	강의 12 시연 10 소프트웨어 실습 23 확인 5분	meeting_schema.json
13:50-14:40	5차시	긴 회의는 문장이 아니라 발화 단위로 나눕니다	강의 12 시연 10 소프트웨어 실습 23 확인 5분	meeting_chunks.json
14:40-15:30	6차시	LangChain은 prompt:model:parser를 하나의 실행 계약으로 묶습니다	강의 12 시연 10 소프트웨어 실습 23 확인 5분	meeting_brief.json
15:30-16:20	7차시	근거가 없는 Action Item은 좋은 문장이 아니라 위험한 업무입니다	강의 12 시연 10 소프트웨어 실습 23 확인 5분	validated_meeting_brief.json
16:20-17:10	8차시	같은 회의 Agent도 도메인별 품질 기준은 달라집니다	강의 12 시연 10 소프트웨어 실습 23 확인 5분	day2_eval_cases.json

11:30-12:00 쉬는 시간 · 12:00-13:00 점심시간 · 17:10-17:40 쉬는 시간 · 17:40-18:00 Q&A·실행 복구

깨진 입력을 모델 문제로 오해하면 복구 순서가 뒤집힌다.

길이·채널·샘플링레이트를 읽고 전사 가능 여부를 판단한다.

이번 차시의 확인 결과: audio_metadata.json

이 차시가 끝나면 세 가지를 설명하고 실행할 수 있습니다

- 01 — 길이·채널·샘플링레이트를 읽고 전사 가능 여부를 판단한다.
- 02 — 파일이 없거나 codec을 열 수 없는데 재시도만 반복한다. 왜 위험한지 설명한다.
- 03 — audio_metadata.json에서 정상과 실패 상태를 직접 확인한다.

처음 보는 용어는 이 네 가지 역할만 구분합니다

용어	이 수업에서 쓰는 뜻
Codec	음성을 저장하는 압축 방식
Container	음성과 메타데이터를 담는 파일 형식
Sample rate	1초를 몇 번 측정했는지
Channel	mono·stereo 같은 음성 통로 수

입력·처리·결과·검증을 분리하면 문제가 빨리 보입니다

INPUT

data/demo_meeting.wav

PROCESS

workspace 경로 확인 → 파일 존재 확인 → WAV
metadata 읽기

OUTPUT

audio_metadata.json

VERIFY

WAV의 길이·rate·channel이 숫자로 반환된다.
workspace 밖 오디오는
WORKSPACE_PATH_BLOCKED로 끝난다.

실행 흐름은 다섯 단계로 고정합니다

01

workspace 경로
확인

02

파일 존재 확인

03

WAV metadata
읽기

04

지원 조건 판정

05

전사 또는 구조화
실패

마지막 판단은 audio_metadata.json에서 사람이 확인합니다.

코드보다 먼저 성공·실패 계약을 정합니다

정상	—— WAV의 길이·rate·channel이 숫자로 반환된다.
경계	—— workspace 밖 오디오는 WORKSPACE_PATH_BLOCKED로 끝난다.
외부 쓰기	—— 기본값 false · dry-run과 사람 승인 뒤에만 별도 adapter 호출
기록	—— status·error_code·입력 식별자·audio_metadata.json

어떤 파일을 열고 어디에서 결과를 확인하는지 먼저 봅시다

입력·fixture

data/demo_meeting.wav

구현 코드

data/demo_meeting_transcript.txt

검증·test

materials/day2/day2_service_lab.ipynb

따라하기 notebook

src/meeting_demo.py

오디오는 먼저 파일 계약부터 확인합니다

깨진 입력을 모델 문제로 오해하면 복구 순서가 뒤집힌다.

FILE_NOT_FOUND와 UNSUPPORTED_AUDIO를 구분하고 변환 또는 fixture 경로를 안내한다.

이 차시에서
버릴 오해

파일이 없거나 codec을
열 수 없는데 재시도만
반복한다.

비슷해 보이는 선택도 책임과 검증 범위가 다릅니다

구분	역할	이번 차시의 판단 기준
Codec	음성을 저장하는 압축 방식	결정론 test로 먼저 확인
Container	음성과 메타데이터를 담는 파일 형식	adapter 경계를 확인
Sample rate	1초를 몇 번 측정했는지	비용·지연·권한을 확인
Channel	mono·stereo 같은 음성 통로 수	사람 판단과 운영 기록을 확인

가장 먼저 재현할 실패는 이것입니다

실패

파일이 없거나 codec을 열 수 없는데
재시도만 반복한다.

사용자 영향

깨진 입력을 모델 문제로 오해하면 복구
순서가 뒤집힌다.

실패가 재현돼야 복구 코드와 회귀 test를 만들 수 있습니다.

복구는 재시도보다 먼저 중단 조건을 정합니다

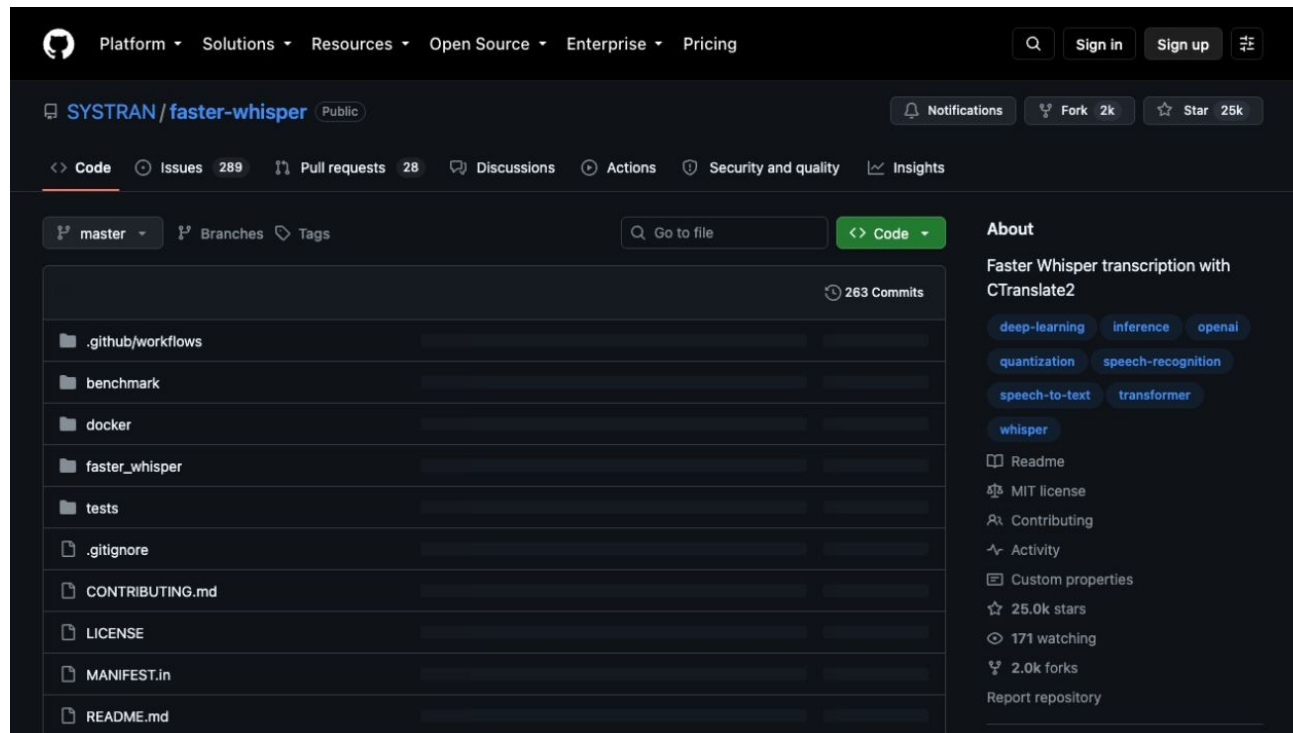
- 01 — 탐지: workspace 밖 오디오는 WORKSPACE_PATH_BLOCKED로 끝난다.
- 02 — 중단: 파일이 없거나 codec을 열 수 없는데 재시도만 반복한다.
- 03 — 복구: FILE_NOT_FOUND와 UNSUPPORTED_AUDIO를 구분하고 변환 또는 fixture 경로를 안내한다.
- 04 — 증거: audio_metadata.json과 test 결과를 함께 남긴다.

Codex는 코드를 대신 쓰는 도구보다 검증 루프에 가깝습니다

오디오 metadata 검사 함수를
작은 책임으로 분리하고 정상·
경로 이탈 test를 추가한다.

저장소 이해

- 허용 범위 변경
- focused test
- diff 리뷰
- 사람 merge



Codex에게는 목표보다 완료 조건을 더 구체적으로 줍니다

목표

오디오 metadata 검사 함수를 작은 책임으로 분리하고 정상·경로 이탈 test를 추가한다.

허용 경로

data/demo_meeting_transcript.txt
materials/day2/day2_service_lab.ipynb

완료 조건

WAV의 길이·rate·channel이 숫자로 반환된다.
workspace 밖 오디오는 WORKSPACE_PATH_BLOCKED로 끝난다.

금지

secret 출력 · workspace 밖 변경 · test 약화 · 사람 승인 없는 외부 쓰기

생성된 patch는 diff와 test 증거를 보고 사람이 결정합니다

- 01 **Scope** — 허용 경로 밖 파일이 바뀌지 않았는가
- 02 **Focused test** — WAV의 길이·rate·channel이 숫자로 반환된다.
- 03 **Boundary test** — workspace 밖 오디오는 WORKSPACE_PATH_BLOCKED로 끝난다.
- 04 **Human merge** — Codex 리뷰와 test 통과는 자동 승인이 아니다

강사가 먼저 정상 경로를 끝까지 실행합니다

- 01 — 열기: `data/demo_meeting.wav`
- 02 — 구현 확인: `data/demo_meeting_transcript.txt`
- 03 — 실행: `python -m src.meeting_demo --help`
- 04 — 성공 신호: WAV의 길이·rate·channel이 숫자로 반환된다.

같은 저장소에서 이 명령을 실행합니다

```
$ python -m src.meeting_demo --help
```

실행 전 확인

현재 경로가 repository root인지 · Python 3.12 환경인지 · .env가 출력되지 않는지

완료 여부는 화면이 아니라 결과 계약으로 확인합니다

STATUS

SUCCESS 또는 EXPECTED_FAILURE

ARTIFACT

audio_metadata.json

사람이 확인할 세 줄

1. WAV의 길이·rate·channel이 숫자로 반환된다.
2. workspace 밖 오디오는 WORKSPACE_PATH_BLOCKED로 끝난다.
3. external_write=false

강사는 실패 한 건도 바로 재현하고 복구합니다

재현

파일이 없거나 codec을 열 수 없는데
재시도만 반복한다.

복구

FILE_NOT_FOUND와
UNSUPPORTED_AUDIO를 구분하고
변환 또는 fixture 경로를 안내한다.

확인: workspace 밖 오디오는 WORKSPACE_PATH_BLOCKED로 끝난다.

이제 같은 코드를 각자 실행하고 한 줄만 바꿉니다

열 파일

src/meeting_demo.py

수정 범위

notebook의 실습 변수 또는 fixture 한 줄 · src 전체를 복사해 바꾸지 않음

완료 기준

WAV의 길이·rate·channel이 숫자로 반환된다.
audio_metadata.json 저장

STEP 1 · 입력과 설정을 확인합니다

파일: data/demo_meeting.wav
변수: 실행 경로·provider·dry-run 여부

결과 파일: audio_metadata.json

STEP 2 · 정상 경로를 실행합니다

```
python -m src.meeting_demo --help
```

성공 신호: WAV의 길이·rate·channel이 숫자로 반환된다.

결과 파일: audio_metadata.json

STEP 3 · 경계값을 바꿔 실패를 확인합니다

workspace 밖 오디오는 WORKSPACE_PATH_BLOCKED로 끝난다.
복구: FILE_NOT_FOUND와 UNSUPPORTED_AUDIO를 구분하고 변환 또는
fixture 경로를 안내한다.

결과에 error_code와 external_write=false가 보이면 완료

정상 case를 focused test로 확인합니다

NORMAL

WAV의 길이·rate·channel이 숫자로 반환된다.

- 01 — 입력 fixture와 실행 command가 기록됐는가
- 02 — 결과 schema가 validate되는가
- 03 — focused test가 실제로 통과했는가

가장 중요한 실패 조건을 test로 고정합니다

BOUNDARY

workspace 밖 오디오는 WORKSPACE_PATH_BLOCKED로 끝난다.

- 01 — raw traceback 대신 stable error_code가 남는가
- 02 — 외부 쓰기·자동 메일이 false인가
- 03 — 실패가 다음 차시에서 재현 가능한가

재직자는 작은 운영 문제 한 곳에 먼저 적용합니다

고객 상담 녹음이 들어오기 전에 포맷과 보존 정책을 검사하는 업로드 단계

- 01 — 실제 업무 데이터 대신 합성·비식별 sample로 먼저 실행
- 02 — 현재 수작업 시간과 오류를 baseline으로 기록
- 03 — 사람 승인 전에는 외부 시스템을 바꾸지 않음
- 04 — audio_metadata.json을 운영 검토 자료로 사용

구직자는 제품 판단과 검증 증거를 함께 보여줍니다

고객 상담 녹음이 들어오기 전에 포맷과 보존 정책을 검사하는 업로드 단계

- 01 — 문제와 사용자 영향을 한 문장으로 설명
- 02 — 정상 demo와 대표 실패 demo를 함께 준비
- 03 — Codex가 만든 diff와 직접 검토한 test 증거를 구분
- 04 — audio_metadata.json과 README 재현 절차를 제출

서비스로 운영하려면 이 네 가지 질문에 답해야 합니다

품질

무엇을 WAV의 길이·rate·channel이 숫자로 반환된다.로 정의하는가

복구

FILE_NOT_FOUND와 UNSUPPORTED_AUDIO를 구분하고 변환 또는 fixture 경로를 안내한다.

안전

어디에서 사람 승인과 external_write=false를 확인하는가

관측

audio_metadata.json에서 어떤 status_error_code를 보는가

1차시를 마치기 전에 세 가지를 확인합니다

- 01 — 설명: 오디오는 먼저 파일 계약부터 확인합니다이 필요한 이유를 한 문장으로 말할 수 있다.
- 02 — 실행: `python -m src.meeting_demo --help`
- 03 — 증거: `audio_metadata.json`과 정상·실패 test 결과가 남아 있다.

다음 차시: faster-whisper는 로컬 STT의 교체 가능한 adapter입니다

faster-whisper는 로컬 STT의 교체 가능한 adapter입니다

CPU int8 환경에서 한국어 음성을 segment와 timestamp로 전사한다.

진행

강의 12분 · 강사 시연 10분 · 소프트웨어
실습 23분 · 실행 확인 5분

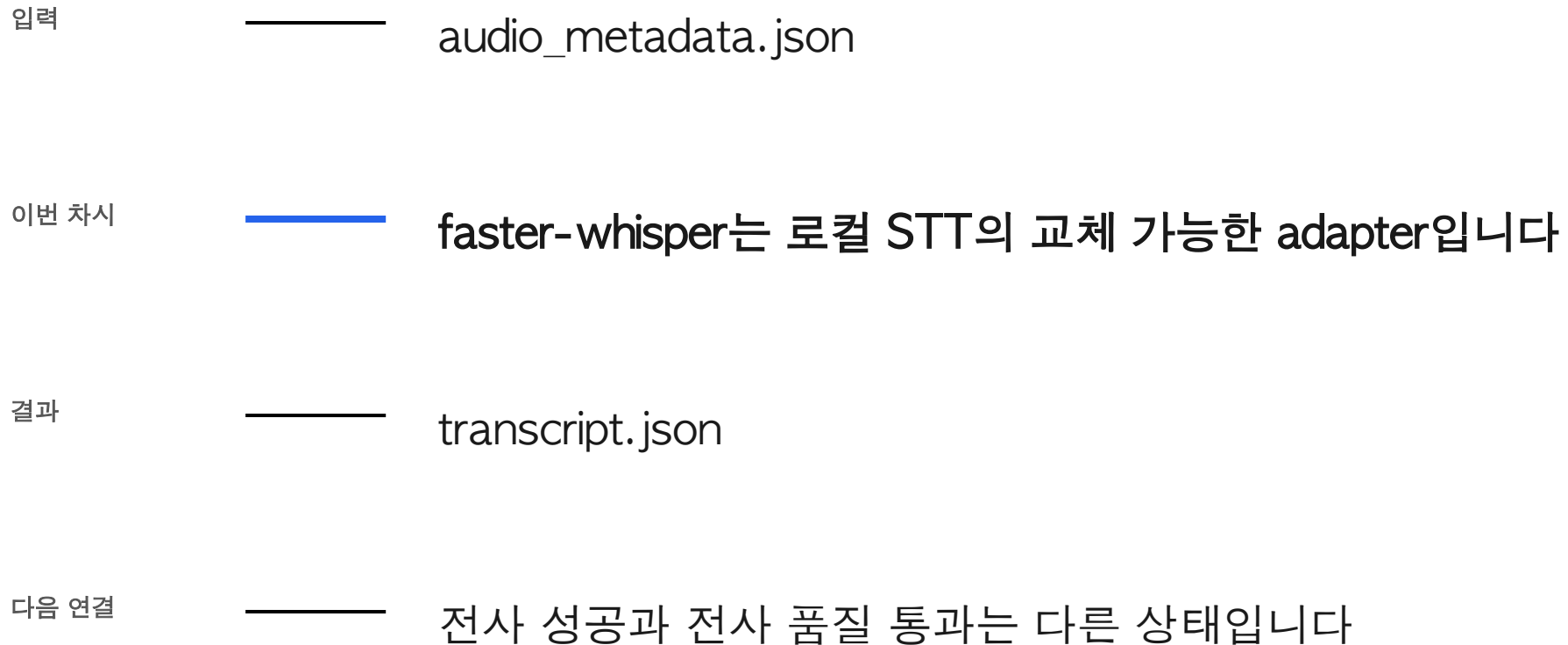
사용 파일

src/meeting_demo.py
requirements-stt-optional.txt

확인할 결과

transcript.json

2차시는 앞의 결과를 이어받아 다음 상태를 만듭니다



모델 호출과 나머지 업무 로직을 분리해야 GPU·모델·provider를 바꿀 수 있다.

CPU int8 환경에서 한국어 음성을 segment와 timestamp로
전사한다.

이번 차시의 확인 결과: transcript.json

이 차시가 끝나면 세 가지를 설명하고 실행할 수 있습니다

- 01 — CPU int8 환경에서 한국어 음성을 segment와 timestamp로 전사한다.
- 02 — 모델 download나 GPU OOM 때문에 후속 수업 전체가 멈춘다. 왜 위험한지 설명한다.
- 03 — transcript.json에서 정상과 실패 상태를 직접 확인한다.

처음 보는 용어는 이 네 가지 역할만 구분합니다

용어	이 수업에서 쓰는 뜻
Whisper	음성을 문자로 바꾸는 모델 계열
faster-whisper	CTranslate2 기반 실행 구현
int8	메모리를 줄이는 계산 형식
Segment	시작·끝 시각이 붙은 발화 조각

입력·처리·결과·검증을 분리하면 문제가 빨리 보입니다

INPUT

src/meeting_demo.py

PROCESS

모델 load → VAD 적용 → beam search

OUTPUT

transcript.json

VERIFY

segment마다 start·end·text가 있고 float가 JSON
직렬화된다.
STT package가 없으면 fixture와
STT_FALLBACK_USED를 함께 남긴다.

실행 흐름은 다섯 단계로 고정합니다

01

모델 load

02

VAD 적용

03

beam search

04

segment 순회

05

JSON 안전 float
변환

마지막 판단은 transcript.json에서 사람이 확인합니다.

코드보다 먼저 성공·실패 계약을 정합니다

정상	—— segment마다 start·end·text가 있고 float가 JSON 직렬화된다.
경계	—— STT package가 없으면 fixture와 STT_FALLBACK_USED를 함께 남긴다.
외부 쓰기	—— 기본값 false · dry-run과 사람 승인 뒤에만 별도 adapter 호출
기록	—— status·error_code·입력 식별자·transcript.json

어떤 파일을 열고 어디에서 결과를 확인하는지 먼저 봅시다

입력·fixture

src/meeting_demo.py

구현 코드

requirements-stt-optional.txt

검증·test

data/demo_meeting.wav

따라하기 notebook

output/day2-stt-lab/transcript.json

faster-whisper는 로컬 STT의 교체 가능한 adapter입니다

모델 호출과 나머지 업무 로직을 분리해야 GPU·모델·provider를 바꿀 수 있다.

local_files_only·CPU int8·작은 모델·reviewed fixture 순서로 내려간다.

이 차시에서
버릴 오해

모델 download나 GPU
OOM 때문에 후속 수업
전체가 멈춘다.

비슷해 보이는 선택도 책임과 검증 범위가 다릅니다

구분	역할	이번 차시의 판단 기준
Whisper	음성을 문자로 바꾸는 모델 계열	결정론 test로 먼저 확인
faster-whisper	CTranslate2 기반 실행 구현	adapter 경계를 확인
int8	메모리를 줄이는 계산 형식	비용·지연·권한을 확인
Segment	시작·끝 시각이 붙은 발화 조각	사람 판단과 운영 기록을 확인

가장 먼저 재현할 실패는 이것입니다

실패

모델 download나 GPU OOM 때문에
후속 수업 전체가 멈춘다.

사용자 영향

모델 호출과 나머지 업무 로직을 분리해야
GPU·모델·provider를 바꿀 수 있다.

실패가 재현돼야 복구 코드와 회귀 test를 만들 수 있습니다.

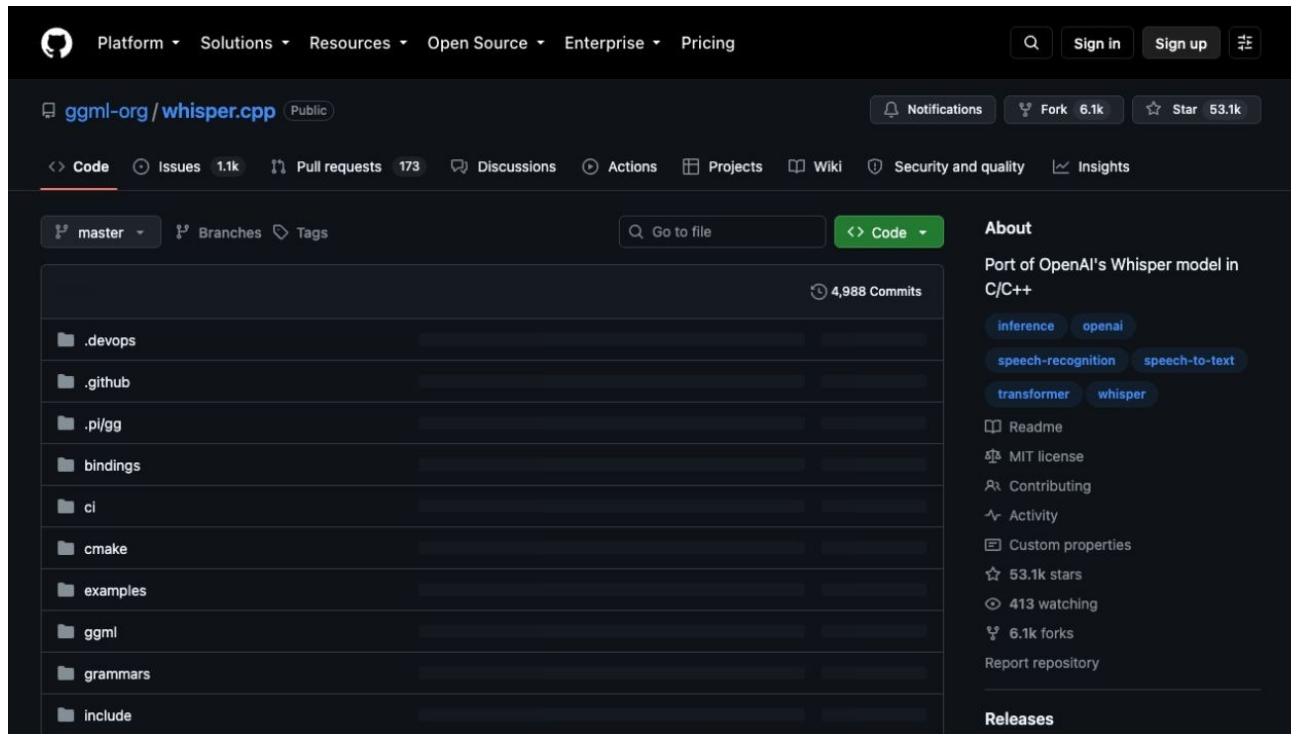
복구는 재시도보다 먼저 중단 조건을 정합니다

- 01 — 탐지: STT package가 없으면 fixture와 STT_FALLBACK_USED를 함께 남긴다.
- 02 — 중단: 모델 download나 GPU OOM 때문에 후속 수업 전체가 멈춘다.
- 03 — 복구: local_files_only·CPU int8·작은 모델·reviewed fixture 순서로 내려간다.
- 04 — 증거: transcript.json과 test 결과를 함께 남긴다.

Codex는 코드를 대신 쓰는 도구보다 검증 루프에 가깝습니다

provider adapter가 같은
transcript contract를 반환하는지
test로 고정한다.

저장소 이해
→ 허용 범위 변경
→ focused test
→ diff 리뷰
→ 사람 merge



Codex에게는 목표보다 완료 조건을 더 구체적으로 줍니다

목표

provider adapter가 같은 transcript contract를 반환하는지 test로 고정한다.

허용 경로

requirements-stt-optional.txt
data/demo_meeting.wav

완료 조건

segment마다 start·end·text가 있고 float가 JSON 직렬화된다.
STT package가 없으면 fixture와 STT_FALLBACK_USED를 함께 남긴다.

금지

secret 출력 · workspace 밖 변경 · test 약화 · 사람 승인 없는 외부 쓰기

생성된 patch는 diff와 test 증거를 보고 사람이 결정합니다

- | | | | |
|----|---------------|----|--|
| 01 | Scope | —— | 허용 경로 밖 파일이 바뀌지 않았는가 |
| 02 | Focused test | —— | segment마다 start·end·text가 있고 float가 JSON 직렬화된다. |
| 03 | Boundary test | —— | STT package가 없으면 fixture와 STT_FALLBACK_USED를 함께 남긴다. |
| 04 | Human merge | —— | Codex 리뷰와 test 통과는 자동 승인이 아니다 |

강사가 먼저 정상 경로를 끝까지 실행합니다

- 01 — 열기: `src/meeting_demo.py`
- 02 — 구현 확인: `requirements-stt-optional.txt`
- 03 — 실행: `python -m src.meeting_demo --audio data/demo_meeting.wav --model small --device cpu --compute-type int8`
- 04 — 성공 신호: segment마다 start·end·text가 있고 float가 JSON 직렬화된다.

같은 저장소에서 이 명령을 실행합니다

```
$ python -m src.meeting_demo --audio data/demo_meeting.wav --model small --device cpu  
--compute-type int8
```

실행 전 확인

현재 경로가 repository root인지 · Python 3.12 환경인지 · .env가 출력되지 않는지

완료 여부는 화면이 아니라 결과 계약으로 확인합니다

STATUS

SUCCESS 또는 EXPECTED_FAILURE

ARTIFACT

transcript.json

사람이 확인할 세 줄

1. segment마다 start·end·text가 있고 float가 JSON 직렬화된다.
2. STT package가 없으면 fixture와 STT_FALLBACK_USED를 함께 남긴다.
3. external_write=false

강사는 실패 한 건도 바로 재현하고 복구합니다

재현

모델 download나 GPU OOM 때문에
후속 수업 전체가 멈춘다.

복구

local_files_only·CPU int8·작은 모델
·reviewed fixture 순서로 내려간다.

확인: STT package가 없으면 fixture와 STT_FALLBACK_USED를 함께 남긴다.

이제 같은 코드를 각자 실행하고 한 줄만 바꿉니다

열 파일

output/day2-stt-lab/transcript.json

수정 범위

notebook의 실습 변수 또는 fixture 한 줄 · src 전체를 복사해 바꾸지 않음

완료 기준

segment마다 start·end·text가 있고 float가 JSON 직렬화된다.
transcript.json 저장

STEP 1 · 입력과 설정을 확인합니다

파일: src/meeting_demo.py
변수: 실행 경로·provider·dry-run 여부

결과 파일: transcript.json

STEP 2 · 정상 경로를 실행합니다

```
python -m src.meeting_demo --audio data/demo_meeting.wav --model small  
--device cpu --compute-type int8
```

성공 신호: segment마다 start·end·text가 있고 float가 JSON 직렬화된다.

결과 파일: transcript.json

STEP 3 · 경계값을 바꿔 실패를 확인합니다

STT package가 없으면 fixture와 STT_FALLBACK_USED를 함께 남긴다.
복구: local_files_only·CPU int8·작은 모델·reviewed fixture 순서로 내려간다.

결과에 error_code와 external_write=false가 보이면 완료

정상 case를 focused test로 확인합니다

NORMAL

segment마다 start·end·text가 있고 float가 JSON 직렬화된다.

- 01 — 입력 fixture와 실행 command가 기록됐는가
- 02 — 결과 schema가 validate되는가
- 03 — focused test가 실제로 통과했는가

가장 중요한 실패 조건을 test로 고정합니다

BOUNDARY

STT package가 없으면 fixture와 STT_FALLBACK_USED를 함께 남긴다.

- 01 — raw traceback 대신 stable error_code가 남는가
- 02 — 외부 쓰기·자동 메일이 false인가
- 03 — 실패가 다음 차시에서 재현 가능한가

재직자는 작은 운영 문제 한 곳에 먼저 적용합니다

사내망에서 원본 음성을 외부로 보내지 않는 상담 품질 분석

- 01 — 실제 업무 데이터 대신 합성·비식별 sample로 먼저 실행
- 02 — 현재 수작업 시간과 오류를 baseline으로 기록
- 03 — 사람 승인 전에는 외부 시스템을 바꾸지 않음
- 04 — transcript.json을 운영 검토 자료로 사용

구직자는 제품 판단과 검증 증거를 함께 보여줍니다

사내망에서 원본 음성을 외부로 보내지 않는 상담 품질 분석

- 01 — 문제와 사용자 영향을 한 문장으로 설명
- 02 — 정상 demo와 대표 실패 demo를 함께 준비
- 03 — Codex가 만든 diff와 직접 검토한 test 증거를 구분
- 04 — transcript.json과 README 재현 절차를 제출

서비스로 운영하려면 이 네 가지 질문에 답해야 합니다

품질

무엇을 segment마다 start·end·text가 있고 float가 JSON 직렬화된다.로 정의하는가

안전

어디에서 사람 승인과 external_write=false를 확인하는가

복구

local_files_only·CPU int8·작은 모델·reviewed fixture 순서로 내려간다.

관측

transcript.json에서 어떤 status·error_code를 보는가

2차시를 마치기 전에 세 가지를 확인합니다

- 01 — 설명: faster-whisper는 로컬 STT의 교체 가능한 adapter입니다. 필요한 이유를 한 문장으로 말할 수 있다.
- 02 — 실행: `python -m src.meeting_demo --audio data/demo_meeting.wav --model small --device cpu --compute-type int8`
- 03 — 증거: transcript.json과 정상·실패 test 결과가 남아 있다.

다음 차시: 전사 성공과 전사 품질 통과는 다른 상태입니다

10:40-11:30 (쉬는 시간 11:30-12:00)

전사 성공과 전사 품질 통과는 다른 상태입니다

무음·반복·낮은 확률·언어 불일치를 품질 flag로 분리한다.

진행

강의 12분 · 강사 시연 10분 · 소프트웨어
실습 23분 · 실행 확인 5분

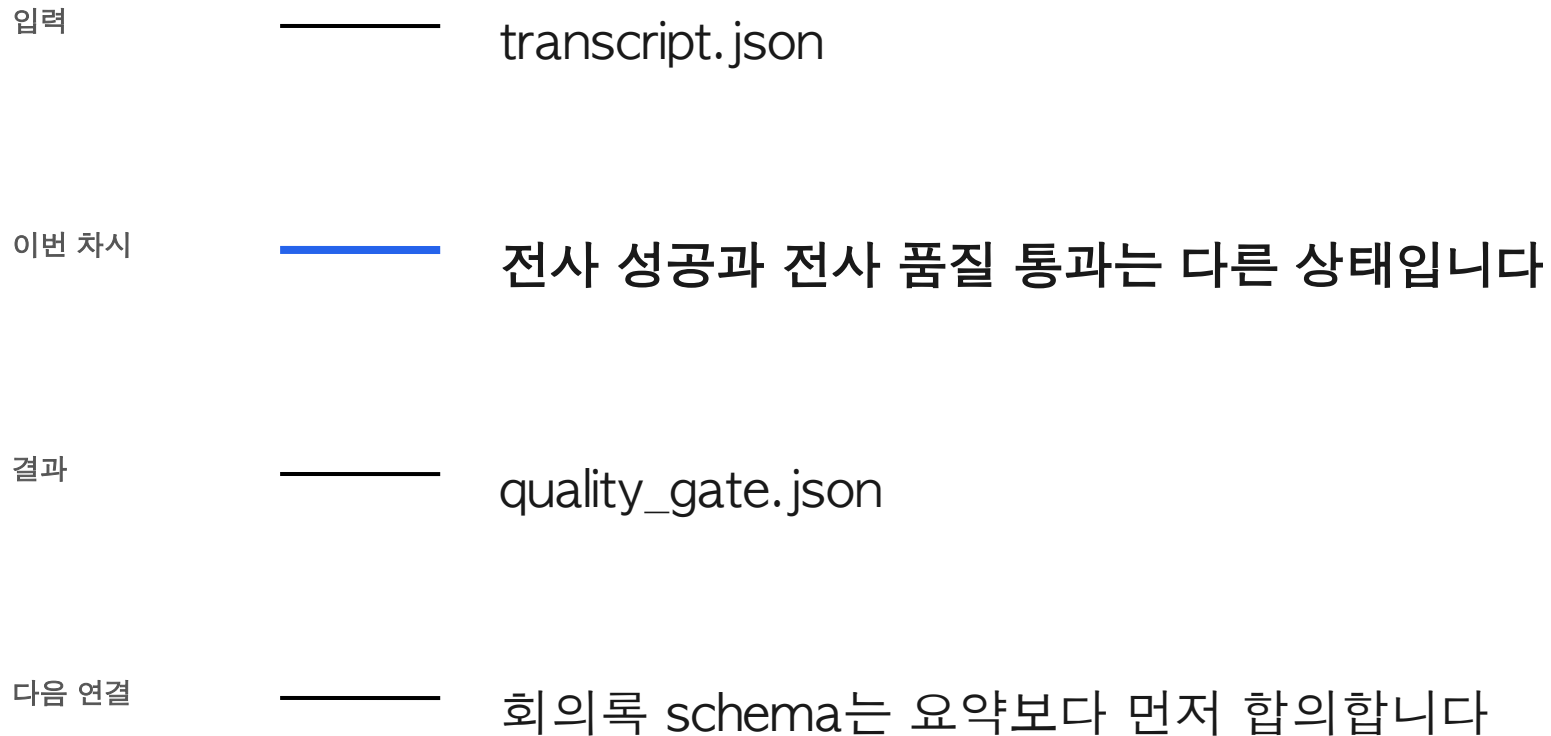
사용 파일

src/meeting_demo.py
tests/test_meeting_agent_workflow.py

확인할 결과

quality_gate.json

3차시는 앞의 결과를 이어받아 다음 상태를 만듭니다



문자가 생성됐다는 사실만으로 회의 결과를 만들면 이름과 숫자 오류가 그대로 번진다.

무음·반복·낮은 확률·언어 불일치를 품질 flag로 분리한다.

이번 차시의 확인 결과: quality_gate.json

이 차시가 끝나면 세 가지를 설명하고 실행할 수 있습니다

- 01 — 무음·반복·낮은 확률·언어 불일치를 품질 flag로 분리한다.
- 02 — 낮은 품질 segment 하나를 전체 성공으로 숨긴다. 왜 위험한지 설명한다.
- 03 — quality_gate.json에서 정상과 실패 상태를 직접 확인한다.

처음 보는 용어는 이 네 가지 역할만 구분합니다

용어	이 수업에서 쓰는 뜻
VAD	말소리 구간을 먼저 찾는 처리
no-speech	해당 구간이 무음일 가능성
log probability	모델이 전사에 가진 상대 신뢰
Hotwords	제품명 등 인식 힌트

입력·처리·결과·검증을 분리하면 문제가 빨리 보입니다

INPUT

src/meeting_demo.py

PROCESS

segment metric 수집 → 반복 탐지 → 언어 확인

OUTPUT

quality_gate.json

VERIFY

한국어·충분한 길이·무 flag일 때 READY다.
fallback·짧은 전사·reference 유사도 저하는 HOLD다.

실행 흐름은 다섯 단계로 고정합니다

01

segment metric
수집

02

반복 탐지

03

언어 확인

04

reference 비교

05

READY 또는 HOLD

마지막 판단은 quality_gate.json에서 사람이 확인합니다.

코드보다 먼저 성공·실패 계약을 정합니다

정상	—— 한국어·충분한 길이·무 flag일 때 READY다.
경계	—— fallback·짧은 전사·reference 유사도 저하는 HOLD다.
외부 쓰기	—— 기본값 false · dry-run과 사람 승인 뒤에만 별도 adapter 호출
기록	—— status·error_code·입력 식별자·quality_gate.json

어떤 파일을 열고 어디에서 결과를 확인하는지 먼저 봅시다

입력·fixture

```
src/meeting_demo.py
```

구현 코드

```
tests/test_meeting_agent_workflow.py
```

검증·test

```
data/demo_meeting_transcript.txt
```

따라하기 notebook

```
output/day2-stt-lab/meeting_result.json
```

전사 성공과 전사 품질 통과는 다른 상태입니다

문자가 생성됐다는 사실만으로 회의 결과를 만들면
이름과 숫자 오류가 그대로 번진다.

flagged segment와 timestamp만 사람에게 보여주고 accept·edit·reject를
받는다.

이 차시에서
버릴 오해

낮은 품질 segment
하나를 전체 성공으로
숨긴다.

비슷해 보이는 선택도 책임과 검증 범위가 다릅니다

구분	역할	이번 차시의 판단 기준
VAD	말소리 구간을 먼저 찾는 처리	결정론 test로 먼저 확인
no-speech	해당 구간이 무음일 가능성	adapter 경계를 확인
log probability	모델이 전사에 가진 상대 신뢰	비용·지연·권한을 확인
Hotwords	제품명 등 인식 힌트	사람 판단과 운영 기록을 확인

가장 먼저 재현할 실패는 이것입니다

실패

낮은 품질 segment 하나를 전체
성공으로 숨긴다.

사용자 영향

문자가 생성됐다는 사실만으로 회의 결과를
만들면 이름과 숫자 오류가 그대로 번진다.

실패가 재현돼야 복구 코드와 회귀 test를 만들 수 있습니다.

복구는 재시도보다 먼저 중단 조건을 정합니다

- 01 — 탐지: fallback·짧은 전사·reference 유사도 저하는 HOLD다.
- 02 — 중단: 낮은 품질 segment 하나를 전체 성공으로 숨긴다.
- 03 — 복구: flagged segment와 timestamp만 사람에게 보여주고 accept·edit·reject를 받는다.
- 04 — 증거: quality_gate.json과 test 결과를 함께 남긴다.

Codex는 코드를 대신 쓰는 도구보다 검증 루프에 가깝습니다

품질 threshold 변경이 기존 정상 음성을 HOLD로 만들지 회귀 test를 작성한다.

저장소 이해

→ 허용 범위 변경

→ focused test

→ diff 리뷰

→ 사람 merge

Google Meet 고객센터

문제를 설명해 주세요.

로그인

Google Meet

Google Meet 시작하기 | 화상 회의 시작 또는 참여하기 | Gemini in Meet | 회의 중 작업 | 회의 종료 후 | **기능 사용 가능 여부** | Google Meet 설정 | Google Meet에서 통화하기

Google Meet 프리미엄 기능 > Google Meet에서 스크립트 사용하기

Google Meet에서 스크립트 사용하기

중요:

- 스크립트 기능은 컴퓨터, 노트북 또는 Android 기기에서만 사용할 수 있습니다.
- 기본적으로 회의 스크립트가 사용 중지되어 있는 학생 라이선스의 Google Workspace for Education 이외에는 모든 Workspace 버전에 회의 스크립트가 기본적으로 사용 설정되어 있습니다. [스크립트 설정에 관해 자세히 알아보기](#)

다음 두 드라이브 모두에 공간이 충분한 경우에만 회의 스크립트를 녹화할 수 있습니다.

- Workspace 조직의 Google Drive
- 회의 호스트의 Drive

[스크립트가 있는 Google Workspace 버전 알아보기](#)

[지원되는 언어 알아보기](#)

Google Meet 프리미엄 기능

- Google Workspace를 위한 프리미엄 Meet 기능
- 화상 회의 녹화하기
- Google Meet에서 스크립트 사용하기**
- Google Meet에서 번역된 자막 사용하기
- Google Meet에서 소그룹 채팅방 사용하기
- Google Meet에서 Calendar 참석자 목록 보기
- Google Meet에서 참여자에게 질문하기
- Google Meet에서 설문조사 진행하기
- Google Meet에서 Slides 공동 발표하기

Codex에게는 목표보다 완료 조건을 더 구체적으로 줍니다

목표

품질 threshold 변경이 기존 정상 음성을 HOLD로 만들지 회귀 test를 작성한다.

허용 경로

tests/test_meeting_agent_workflow.py
data/demo_meeting_transcript.txt

완료 조건

한국어·충분한 길이·무 flag일 때 READY다.
fallback·짧은 전사·reference 유사도 저하는 HOLD다.

금지

secret 출력 · workspace 밖 변경 · test 약화 · 사람 승인 없는 외부 쓰기

생성된 patch는 diff와 test 증거를 보고 사람이 결정합니다

- 01 **Scope** — 허용 경로 밖 파일이 바뀌지 않았는가
- 02 **Focused test** — 한국어·충분한 길이·무 flag일 때 READY다.
- 03 **Boundary test** — fallback·짧은 전사·reference 유사도 저하는 HOLD다.
- 04 **Human merge** — Codex 리뷰와 test 통과는 자동 승인이 아니다

강사가 먼저 정상 경로를 끝까지 실행합니다

- 01 — 열기: `src/meeting_demo.py`
- 02 — 구현 확인: `tests/test_meeting_agent_workflow.py`
- 03 — 실행: `python -m pytest -q tests/test_meeting_agent_workflow.py`
- 04 — 성공 신호: 한국어·충분한 길이·무 flag일 때 READY다.

같은 저장소에서 이 명령을 실행합니다

```
$ python -m pytest -q tests/test_meeting_agent_workflow.py
```

실행 전 확인

현재 경로가 repository root인지 · Python 3.12 환경인지 · .env가 출력되지 않는지

완료 여부는 화면이 아니라 결과 계약으로 확인합니다

STATUS

SUCCESS 또는 EXPECTED_FAILURE

ARTIFACT

quality_gate.json

사람이 확인할 세 줄

1. 한국어·충분한 길이·무 flag일 때 READY다.
2. fallback·짧은 전사·reference 유사도 저하는 HOLD다.
3. external_write=false

강사는 실패 한 건도 바로 재현하고 복구합니다

재현

낮은 품질 segment 하나를 전체 성공으로 숨긴다.

복구

flagged segment와 timestamp만 사람에게 보여주고 accept·edit·reject를 받는다.

확인: fallback·짧은 전사·reference 유사도 저하는 HOLD다.

이제 같은 코드를 각자 실행하고 한 줄만 바꿉니다

열 파일

output/day2-stt-lab/meeting_result.json

수정 범위

notebook의 실습 변수 또는 fixture 한 줄 · src 전체를 복사해 바꾸지 않음

완료 기준

한국어·충분한 길이·무 flag일 때 READY다.
quality_gate.json 저장

STEP 1 · 입력과 설정을 확인합니다

파일: src/meeting_demo.py

변수: 실행 경로·provider·dry-run 여부

결과 파일: quality_gate.json

STEP 2 · 정상 경로를 실행합니다

```
python -m pytest -q tests/test_meeting_agent_workflow.py
```

성공 신호: 한국어·충분한 길이·무 flag일 때 READY다.

결과 파일: quality_gate.json

STEP 3 · 경계값을 바꿔 실패를 확인합니다

fallback·짧은 전사·reference 유사도 저하는 HOLD다.

복구: flagged segment와 timestamp만 사람에게 보여주고 accept·edit·reject를 받는다.

결과에 error_code와 external_write=false가 보이면 완료

정상 case를 focused test로 확인합니다

NORMAL

한국어·충분한 길이·무 flag일 때 READY다.

- 01 — 입력 fixture와 실행 command가 기록됐는가
- 02 — 결과 schema가 validate되는가
- 03 — focused test가 실제로 통과했는가

가장 중요한 실패 조건을 test로 고정합니다

BOUNDARY

fallback·짧은 전사·reference 유사도 저하는 HOLD다.

- 01 — raw traceback 대신 stable error_code가 남는가
- 02 — 외부 쓰기·자동 메일이 false인가
- 03 — 실패가 다음 차시에서 재현 가능한가

재직자는 작은 운영 문제 한 곳에 먼저 적용합니다

회의록 생성 전에 고유명사와 금액 구간만 운영자가 spot check

- 01 — 실제 업무 데이터 대신 합성·비식별 sample로 먼저 실행
- 02 — 현재 수작업 시간과 오류를 baseline으로 기록
- 03 — 사람 승인 전에는 외부 시스템을 바꾸지 않음
- 04 — quality_gate.json을 운영 검토 자료로 사용

구직자는 제품 판단과 검증 증거를 함께 보여줍니다

회의록 생성 전에 고유명사와 금액 구간만 운영자가 spot check

- 01 — 문제와 사용자 영향을 한 문장으로 설명
- 02 — 정상 demo와 대표 실패 demo를 함께 준비
- 03 — Codex가 만든 diff와 직접 검토한 test 증거를 구분
- 04 — quality_gate.json과 README 재현 절차를 제출

서비스로 운영하려면 이 네 가지 질문에 답해야 합니다

품질

무엇을 한국어·충분한 길이·무 flag일 때 READY다.로 정의하는가

복구

flagged segment와 timestamp만 사람에게 보여주고 accept·edit·reject를 받는다.

안전

어디에서 사람 승인과 external_write=false를 확인하는가

관측

quality_gate.json에서 어떤 status·error_code를 보는가

3차시를 마치기 전에 세 가지를 확인합니다

- 01 — 설명: 전사 성공과 전사 품질 통과는 다른 상태입니다. 필요한 이유를 한 문장으로 말할 수 있다.
- 02 — 실행: `python -m pytest -q tests/test_meeting_agent_workflow.py`
- 03 — 증거: `quality_gate.json`과 정상·실패 test 결과가 남아 있다.

다음 차시: 회의록 schema는 요약보다 먼저 합의합니다

회의록 schema는 요약보다 먼저 합의합니다

summary·decision·action item·evidence를 Pydantic 계약으로 만든다.

진행

강의 12분 · 강사 시연 10분 · 소프트웨어
실습 23분 · 실행 확인 5분

사용 파일

src/langchain_lab.py
src/course_services/meeting_service.py

확인할 결과

meeting_schema.json

4차시는 앞의 결과를 이어받아 다음 상태를 만듭니다

입력	———	quality_gate.json
이번 차시	———	회의록 schema는 요약보다 먼저 합의합니다
결과	———	meeting_schema.json
다음 연결	———	긴 회의는 문장이 아니라 발화 단위로 나눕니다

자유로운 문장은 읽기 좋지만 downstream 자동화가 실패해도 원인을 찾기 어렵다.

summary·decision·action item·evidence를 Pydantic 계약으로 만든다.

이번 차시의 확인 결과: meeting_schema.json

이 차시가 끝나면 세 가지를 설명하고 실행할 수 있습니다

- 01 — summary·decision·action item·evidence를 Pydantic 계약으로 만든다.
- 02 — 원문에 없는 담당자와 날짜를 빈칸 대신 추측한다. 왜 위험한지 설명한다.
- 03 — meeting_schema.json에서 정상과 실패 상태를 직접 확인한다.

처음 보는 용어는 이 네 가지 역할만 구분합니다

용어	이 수업에서 쓰는 뜻
Schema	결과 필드와 형식의 약속
Pydantic	Python 데이터 검증 도구
Structured Output	정해진 구조로 받는 모델 결과
null	원문에 값이 없음을 정직하게 표시

입력·처리·결과·검증을 분리하면 문제가 빨리 보입니다

INPUT

src/langchain_lab.py

PROCESS

필수 필드 선언 → type·범위 검사 → 추가 필드 차단

OUTPUT

meeting_schema.json

VERIFY

세 개 Action Item이 evidence_ids와 함께 validate된다.
automatic_email=true 또는 evidence 누락은 validation error다.

실행 흐름은 다섯 단계로 고정합니다

01

필수 필드 선언

02

type·범위 검사

03

추가 필드 차단

04

evidence 확인

05

validation error 반환

마지막 판단은 meeting_schema.json에서 사람이 확인합니다.

코드보다 먼저 성공·실패 계약을 정합니다

- 정상 — 세 개 Action Item이 evidence_ids와 함께 validate된다.
- 경계 — automatic_email=true 또는 evidence 누락은 validation error다.
- 외부 쓰기 — 기본값 false · dry-run과 사람 승인 뒤에만 별도 adapter 호출
- 기록 — status·error_code·입력 식별자·meeting_schema.json

어떤 파일을 열고 어디에서 결과를 확인하는지 먼저 봅시다

입력·fixture

```
src/langchain_lab.py
```

구현 코드

```
src/course_services/meeting_service.py
```

검증·test

```
tests/test_langchain_langgraph_lab.py
```

따라하기 notebook

```
materials/day2/day2_service_lab.ipynb
```

회의록 schema는 요약보다 먼저 합의합니다

자유로운 문장은 읽기 좋지만 downstream 자동화가
실패해도 원인을 찾기 어렵다.

미상값은 null로 두고 evidence가 없는 Action Item은 HOLD한다.

이 차시에서
버릴 오해

원문에 없는 담당자와
날짜를 빈칸 대신
추측한다.

비슷해 보이는 선택도 책임과 검증 범위가 다릅니다

구분	역할	이번 차시의 판단 기준
Schema	결과 필드와 형식의 약속	결정론 test로 먼저 확인
Pydantic	Python 데이터 검증 도구	adapter 경계를 확인
Structured Output	정해진 구조로 받는 모델 결과	비용·지연·권한을 확인
null	원문에 값이 없음을 정직하게 표시	사람 판단과 운영 기록을 확인

가장 먼저 재현할 실패는 이것입니다

실패

원문에 없는 담당자와 날짜를 빈칸 대신 추측한다.

사용자 영향

자유로운 문장은 읽기 좋지만 downstream 자동화가 실패해도 원인을 찾기 어렵다.

실패가 재현돼야 복구 코드와 회귀 test를 만들 수 있습니다.

복구는 재시도보다 먼저 중단 조건을 정합니다

- 01 — 탐지: automatic_email=true 또는 evidence 누락은 validation error다.
- 02 — 중단: 원문에 없는 담당자와 날짜를 빈칸 대신 추측한다.
- 03 — 복구: 이상값은 null로 두고 evidence가 없는 Action Item은 HOLD한다.
- 04 — 증거: meeting_schema.json과 test 결과를 함께 남긴다.

Codex는 코드를 대신 쓰는 도구보다 검증 루프에 가깝습니다

회의 schema에 필드를 추가하고
extra=forbid·누락 field test를
함께 만든다.

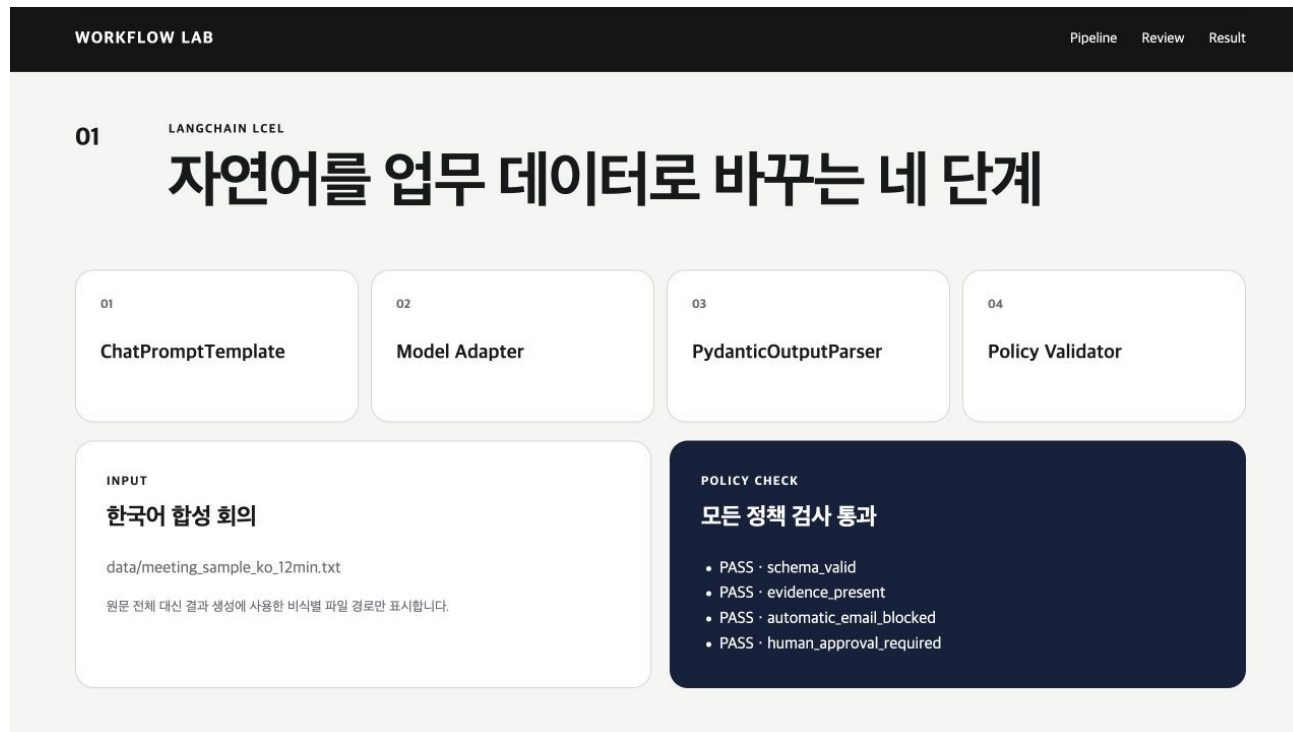
저장소 이해

→ 허용 범위 변경

→ focused test

→ diff 리뷰

→ 사람 merge



Codex에게는 목표보다 완료 조건을 더 구체적으로 줍니다

목표

회의 schema에 필드를 추가하고 extra=forbid-누락 field test를 함께 만든다.

허용 경로

src/course_services/meeting_service.py
tests/test_langchain_langgraph_lab.py

완료 조건

세 개 Action Item이 evidence_ids와 함께 validate된다.
automatic_email=true 또는 evidence 누락은 validation error다.

금지

secret 출력 · workspace 밖 변경 · test 약화 · 사람 승인 없는 외부 쓰기

생성된 patch는 diff와 test 증거를 보고 사람이 결정합니다

- 01 **Scope** — 허용 경로 밖 파일이 바뀌지 않았는가
- 02 **Focused test** — 세 개 Action Item이 evidence_ids와 함께 validate된다.
- 03 **Boundary test** — automatic_email=true 또는 evidence 누락은 validation error다.
- 04 **Human merge** — Codex 리뷰와 test 통과는 자동 승인이 아니다

강사가 먼저 정상 경로를 끝까지 실행합니다

- 01 — 열기: `src/langchain_lab.py`
- 02 — 구현 확인: `src/course_services/meeting_service.py`
- 03 — 실행: `python -m src.langchain_lab --provider fixture`
- 04 — 성공 신호: 세 개 Action Item이 `evidence_ids`와 함께 `validate`된다.

같은 저장소에서 이 명령을 실행합니다

```
$ python -m src.langchain_lab --provider fixture
```

실행 전 확인

현재 경로가 repository root인지 · Python 3.12 환경인지 · .env가 출력되지 않는지

완료 여부는 화면이 아니라 결과 계약으로 확인합니다

STATUS

SUCCESS 또는 EXPECTED_FAILURE

ARTIFACT

meeting_schema.json

사람이 확인할 세 줄

1. 세 개 Action Item이 evidence_ids와 함께 validate된다.
2. automatic_email=true 또는 evidence 누락은 validation error다.
3. external_write=false

강사는 실패 한 건도 바로 재현하고 복구합니다

재현

원문에 없는 담당자와 날짜를 빈칸 대신 추측한다.

복구

미상값은 null로 두고 evidence가 없는 Action Item은 HOLD한다.

확인: automatic_email=true 또는 evidence 누락은 validation error다.

이제 같은 코드를 각자 실행하고 한 줄만 바꿉니다

열 파일

materials/day2/day2_service_lab.ipynb

수정 범위

notebook의 실습 변수 또는 fixture 한 줄 · src 전체를 복사해 바꾸지 않음

완료 기준

세 개 Action Item이 evidence_ids와 함께 validate된다.
meeting_schema.json 저장

STEP 1 · 입력과 설정을 확인합니다

파일: `src/langchain_lab.py`
변수: 실행 경로·provider·dry-run 여부

결과 파일: `meeting_schema.json`

STEP 2 · 정상 경로를 실행합니다

```
python -m src.langchain_lab --provider fixture
```

성공 신호: 세 개 Action Item이 evidence_ids와 함께 validate된다.

결과 파일: meeting_schema.json

STEP 3 · 경계값을 바꿔 실패를 확인합니다

automatic_email=true 또는 evidence 누락은 validation error다.
복구: 미상값은 null로 두고 evidence가 없는 Action Item은 HOLD한다.

결과에 error_code와 external_write=false가 보이면 완료

정상 case를 focused test로 확인합니다

NORMAL

세 개 Action Item이 evidence_ids와 함께 validate된다.

- 01 — 입력 fixture와 실행 command가 기록됐는가
- 02 — 결과 schema가 validate되는가
- 03 — focused test가 실제로 통과했는가

가장 중요한 실패 조건을 test로 고정합니다

BOUNDARY

`automatic_email=true` 또는 `evidence` 누락은 `validation error`다.

- 01 — raw traceback 대신 `stable error_code`가 남는가
- 02 — 외부 쓰기·자동 메일이 `false`인가
- 03 — 실패가 다음 차시에서 재현 가능한가

재직자는 작은 운영 문제 한 곳에 먼저 적용합니다

회의 요약을 Jira·Notion·메일로 보내기 전 공통 계약

- 01 — 실제 업무 데이터 대신 합성·비식별 sample로 먼저 실행
- 02 — 현재 수작업 시간과 오류를 baseline으로 기록
- 03 — 사람 승인 전에는 외부 시스템을 바꾸지 않음
- 04 — meeting_schema.json을 운영 검토 자료로 사용

구직자는 제품 판단과 검증 증거를 함께 보여줍니다

회의 요약을 Jira·Notion·메일로 보내기 전 공통 계약

- 01 — 문제와 사용자 영향을 한 문장으로 설명
- 02 — 정상 demo와 대표 실패 demo를 함께 준비
- 03 — Codex가 만든 diff와 직접 검토한 test 증거를 구분
- 04 — meeting_schema.json과 README 재현 절차를 제출

서비스로 운영하려면 이 네 가지 질문에 답해야 합니다

품질

무엇을 세 개 Action Item이 evidence_ids와 함께 validate된다.로 정의하는가

안전

어디에서 사람 승인과 external_write=false를 확인하는가

복구

미상값은 null로 두고 evidence가 없는 Action Item은 HOLD한다.

관측

meeting_schema.json에서 어떤 status_error_code를 보는가

4차시를 마치기 전에 세 가지를 확인합니다

- 01 — 설명: 회의록 schema는 요약보다 먼저 합의합시다이 필요한 이유를 한 문장으로 말할 수 있다.
- 02 — 실행: `python -m src.langchain_lab --provider fixture`
- 03 — 증거: `meeting_schema.json`과 정상·실패 test 결과가 남아 있다.

다음 차시: 긴 회의는 문장이 아니라 발화 단위로 나눕니다

13:50-14:40 (쉬는 시간 17:10-17:40)

긴 회의는 문장이 아니라 발화 단위로 나눕니다

timestamp와 evidence ID를 잃지 않는 chunk를 만든다.

진행

강의 12분 · 강사 시연 10분 · 소프트웨어
실습 23분 · 실행 확인 5분

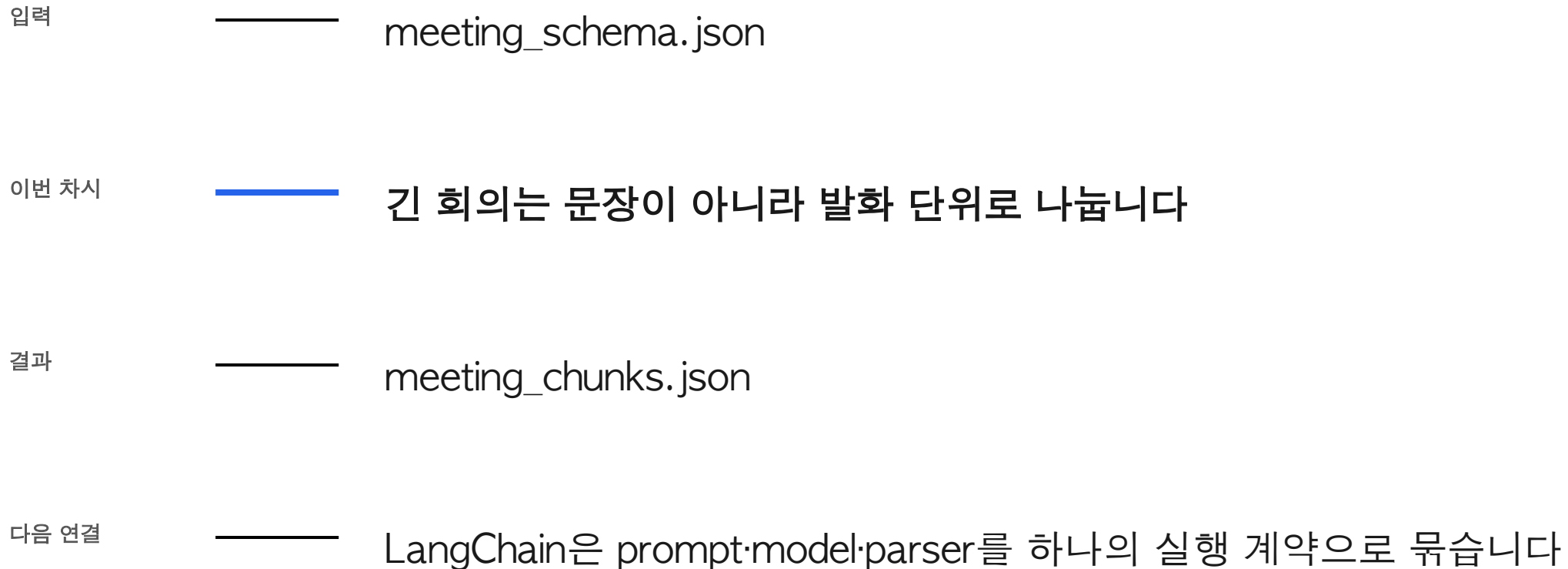
사용 파일

src/course_services/meeting_service.py
tests/test_course_services.py

확인할 결과

meeting_chunks.json

5차시는 앞의 결과를 이어받아 다음 상태를 만듭니다



글자 수만 자르면 담당자·결정·근거가 서로 다른 chunk로 갈라진다.

timestamp와 evidence ID를 잃지 않는 chunk를 만든다.

이번 차시의 확인 결과: meeting_chunks.json

이 차시가 끝나면 세 가지를 설명하고 실행할 수 있습니다

- 01 — timestamp와 evidence ID를 잃지 않는 chunk를 만든다.
- 02 — overlap 때문에 같은 Action Item이 두 번 생성된다. 왜 위험한지 설명한다.
- 03 — meeting_chunks.json에서 정상과 실패 상태를 직접 확인한다.

처음 보는 용어는 이 네 가지 역할만 구분합니다

용어	이 수업에서 쓰는 뜻
Chunk	모델에 넣을 작은 입력 묶음
Overlap	경계 문맥을 이어 주는 중복 구간
Map	chunk별 초안을 만드는 단계
Reduce	여러 초안을 하나로 합치는 단계

입력·처리·결과·검증을 분리하면 문제가 빨리 보입니다

INPUT

src/course_services/meeting_service.py

OUTPUT

meeting_chunks.json

PROCESS

segment 정렬 → 길이 누적 → 경계에서 close

VERIFY

모든 segment ID가 적어도 한 chunk에 존재한다.
max_chars<40과 음수 overlap은 명시 오류다.

실행 흐름은 다섯 단계로 고정합니다

01

segment 정렬

02

길이 누적

03

경계에서 close

04

한 segment overlap

05

chunk ID 저장

마지막 판단은 meeting_chunks.json에서 사람이 확인합니다.

코드보다 먼저 성공·실패 계약을 정합니다

정상	—— 모든 segment ID가 적어도 한 chunk에 존재한다.
경계	—— max_chars<40과 음수 overlap은 명시 오류다.
외부 쓰기	—— 기본값 false · dry-run과 사람 승인 뒤에만 별도 adapter 호출
기록	—— status·error_code·입력 식별자·meeting_chunks.json

어떤 파일을 열고 어디에서 결과를 확인하는지 먼저 봅시다

입력·fixture

```
src/course_services/meeting_service.py
```

구현 코드

```
tests/test_course_services.py
```

검증·test

```
data/meeting_sample_ko_12min.txt
```

따라하기 notebook

```
materials/day2/day2_service_lab.ipynb
```

긴 회의는 문장이 아니라 발화 단위로 나눕니다

글자 수만 자르면 담당자·결정·근거가 서로 다른 chunk로 갈라진다.

evidence ID 기반 deduplication과 최종 사람 검토를 적용한다.

이 차시에서
버릴 오해

overlap 때문에 같은
Action Item이 두 번
생성된다.

비슷해 보이는 선택도 책임과 검증 범위가 다릅니다

구분	역할	이번 차시의 판단 기준
Chunk	모델에 넣을 작은 입력 묶음	결정론 test로 먼저 확인
Overlap	경계 문맥을 이어 주는 중복 구간	adapter 경계를 확인
Map	chunk별 초안을 만드는 단계	비용·지연·권한을 확인
Reduce	여러 초안을 하나로 합치는 단계	사람 판단과 운영 기록을 확인

가장 먼저 재현할 실패는 이것입니다

실패

overlap 때문에 같은 Action Item이
두 번 생성된다.

사용자 영향

글자 수만 자르면 담당자·결정·근거가 서로
다른 chunk로 갈라진다.

실패가 재현돼야 복구 코드와 회귀 test를 만들 수 있습니다.

복구는 재시도보다 먼저 중단 조건을 정합니다

- 01 — 탐지: max_chars<40과 음수 overlap은 명시 오류다.
- 02 — 중단: overlap 때문에 같은 Action Item이 두 번 생성된다.
- 03 — 복구: evidence ID 기반 deduplication과 최종 사람 검토를 적용한다.
- 04 — 증거: meeting_chunks.json과 test 결과를 함께 남긴다.

Codex는 코드를 대신 쓰는 도구보다 검증 루프에 가깝습니다

segment 경계를 보존하는 chunk
함수와 작은 max_chars failure
test를 작성한다.

저장소 이해

→ 허용 범위 변경

→ focused test

→ diff 리뷰

→ 사람 merge



Loading JupyterLite...

Codex에게는 목표보다 완료 조건을 더 구체적으로 줍니다

목표

segment 경계를 보존하는 chunk 함수와 작은 max_chars failure test를 작성한다.

허용 경로

tests/test_course_services.py
data/meeting_sample_ko_12min.txt

완료 조건

모든 segment ID가 적어도 한 chunk에 존재한다.
max_chars < 40과 음수 overlap은 명시 오류다.

금지

secret 출력 · workspace 밖 변경 · test 약화 · 사람 승인 없는 외부 쓰기

생성된 patch는 diff와 test 증거를 보고 사람이 결정합니다

- 01 **Scope** — 허용 경로 밖 파일이 바뀌지 않았는가
- 02 **Focused test** — 모든 segment ID가 적어도 한 chunk에 존재한다.
- 03 **Boundary test** — max_chars<40과 음수 overlap은 명시 오류다.
- 04 **Human merge** — Codex 리뷰와 test 통과는 자동 승인이 아니다

강사가 먼저 정상 경로를 끝까지 실행합니다

- 01 — 열기: `src/course_services/meeting_service.py`
- 02 — 구현 확인: `tests/test_course_services.py`
- 03 — 실행: `python -m pytest -q tests/test_course_services.py -k meeting_chunks`
- 04 — 성공 신호: 모든 segment ID가 적어도 한 chunk에 존재한다.

같은 저장소에서 이 명령을 실행합니다

```
$ python -m pytest -q tests/test_course_services.py -k meeting_chunks
```

실행 전 확인

현재 경로가 repository root인지 · Python 3.12 환경인지 · .env가 출력되지 않는지

완료 여부는 화면이 아니라 결과 계약으로 확인합니다

STATUS

SUCCESS 또는 EXPECTED_FAILURE

ARTIFACT

meeting_chunks.json

사람이 확인할 세 줄

1. 모든 segment ID가 적어도 한 chunk에 존재한다.
2. max_chars<40과 음수 overlap은 명시 오류다.
3. external_write=false

강사는 실패 한 건도 바로 재현하고 복구합니다

재현

overlap 때문에 같은 Action Item이 두 번 생성된다.

복구

evidence ID 기반 deduplication과 최종 사람 검토를 적용한다.

확인: max_chars<40과 음수 overlap은 명시 오류다.

이제 같은 코드를 각자 실행하고 한 줄만 바꿉니다

열 파일

materials/day2/day2_service_lab.ipynb

수정 범위

notebook의 실습 변수 또는 fixture 한 줄 · src 전체를 복사해 바꾸지 않음

완료 기준

모든 segment ID가 적어도 한 chunk에 존재한다.
meeting_chunks.json 저장

STEP 1 · 입력과 설정을 확인합니다

파일: `src/course_services/meeting_service.py`
변수: 실행 경로·provider·dry-run 여부

결과 파일: `meeting_chunks.json`

STEP 2 · 정상 경로를 실행합니다

```
python -m pytest -q tests/test_course_services.py -k meeting_chunks
```

성공 신호: 모든 segment ID가 적어도 한 chunk에 존재한다.

결과 파일: meeting_chunks.json

STEP 3 · 경계값을 바꿔 실패를 확인합니다

`max_chars < 40`과 음수 `overlap`은 명시 오류다.
복구: evidence ID 기반 deduplication과 최종 사람 검토를 적용한다.

결과에 `error_code`와 `external_write=false`가 보이면 완료

정상 case를 focused test로 확인합니다

NORMAL

모든 segment ID가 적어도 한 chunk에 존재한다.

- 01 — 입력 fixture와 실행 command가 기록됐는가
- 02 — 결과 schema가 validate되는가
- 03 — focused test가 실제로 통과했는가

가장 중요한 실패 조건을 test로 고정합니다

BOUNDARY

`max_chars < 40`과 음수 `overlap`은 명시 오류다.

- 01 — raw traceback 대신 stable error_code가 남는가
- 02 — 외부 쓰기·자동 메일이 false인가
- 03 — 실패가 다음 차시에서 재현 가능한가

재직자는 작은 운영 문제 한 곳에 먼저 적용합니다

10분 이상 상품 운영 회의를 결정·담당자·기한으로 축약

- 01 — 실제 업무 데이터 대신 합성·비식별 sample로 먼저 실행
- 02 — 현재 수작업 시간과 오류를 baseline으로 기록
- 03 — 사람 승인 전에는 외부 시스템을 바꾸지 않음
- 04 — meeting_chunks.json을 운영 검토 자료로 사용

구직자는 제품 판단과 검증 증거를 함께 보여줍니다

10분 이상 상품 운영 회의를 결정·담당자·기한으로 축약

- 01 — 문제와 사용자 영향을 한 문장으로 설명
- 02 — 정상 demo와 대표 실패 demo를 함께 준비
- 03 — Codex가 만든 diff와 직접 검토한 test 증거를 구분
- 04 — meeting_chunks.json과 README 재현 절차를 제출

서비스로 운영하려면 이 네 가지 질문에 답해야 합니다

품질

무엇을 모든 segment ID가 적어도 한 chunk에 존재한다.로 정의하는가

복구

evidence ID 기반 deduplication과 최종 사람 검토를 적용한다.

안전

어디에서 사람 승인과 external_write=false를 확인하는가

관측

meeting_chunks.json에서 어떤 status_error_code를 보는가

5차시를 마치기 전에 세 가지를 확인합니다

- 01 — 설명: 긴 회의는 문장이 아니라 발화 단위로 나뉩니다이 필요한 이유를 한 문장으로 말할 수 있다.
- 02 — 실행: `python -m pytest -q tests/test_course_services.py -k meeting_chunks`
- 03 — 증거: `meeting_chunks.json`과 정상·실패 test 결과가 남아 있다.

다음 차시: LangChain은 `prompt-model-parser`를 하나의 실행 계약으로 묶습니다

14:40-15:30 (쉬는 시간 17:10-17:40)

LangChain은 prompt·model·parser를 하나의 실행 계약으로 묶습니다

fixture·Ollama·OpenAI가 같은 MeetingBrief를 반환하게 한다.

진행

강의 12분 · 강사 시연 10분 · 소프트웨어
실습 23분 · 실행 확인 5분

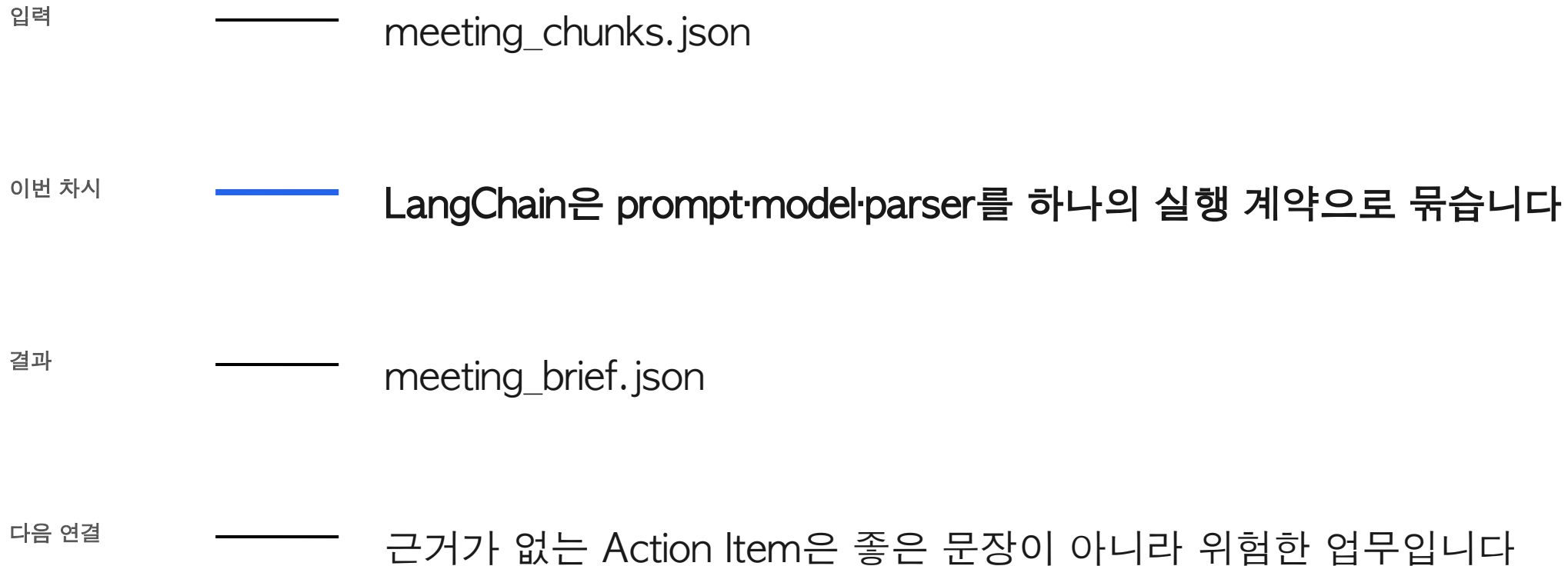
사용 파일

src/langchain_lab.py
src/openai_provider.py

확인할 결과

meeting_brief.json

6차시는 앞의 결과를 이어받아 다음 상태를 만듭니다



provider별 코드를 업무 로직에 섞으면 모델 변경이 곧 전체 재개발이 된다.

fixture·Ollama·OpenAI가 같은 MeetingBrief를 반환하게 한다.

이번 차시의 확인 결과: meeting_brief.json

이 차시가 끝나면 세 가지를 설명하고 실행할 수 있습니다

- 01 — fixture·Ollama·OpenAI가 같은 MeetingBrief를 반환하게 한다.
- 02 — OpenAI 오류가 조용히 fixture 성공처럼 보인다. 왜 위험한지 설명한다.
- 03 — meeting_brief.json에서 정상과 실패 상태를 직접 확인한다.

처음 보는 용어는 이 네 가지 역할만 구분합니다

용어	이 수업에서 쓰는 뜻
Runnable	같은 invoke 계약으로 실행되는 구성요소
LCEL	Runnable을 연결하는 표현 방식
Adapter	provider 차이를 흡수하는 층
Fallback	실패 시 정해진 대체 경로

입력·처리·결과·검증을 분리하면 문제가 빨리 보입니다

INPUT

src/langchain_lab.py

PROCESS

prompt 구성 → provider 선택 → structured generation

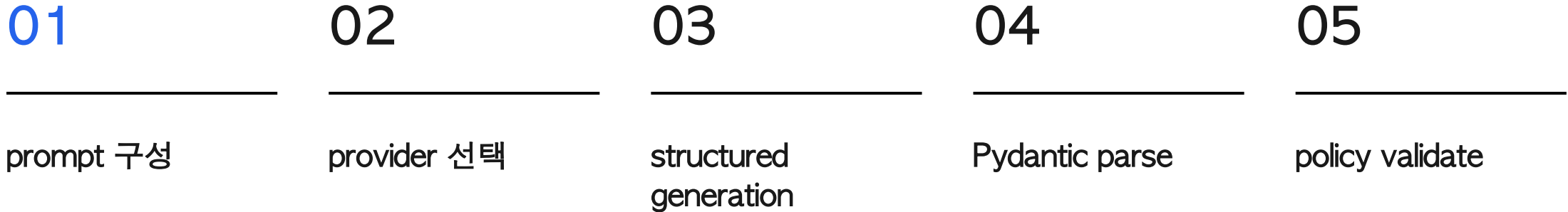
OUTPUT

meeting_brief.json

VERIFY

Qwen과 fixture가 같은 필수 field를 반환한다.
provider 실패 시 허용 여부에 따라 fallback 또는 구조화 오류가 난다.

실행 흐름은 다섯 단계로 고정합니다



마지막 판단은 meeting_brief.json에서 사람이 확인합니다.

코드보다 먼저 성공·실패 계약을 정합니다

- 정상 — Qwen과 fixture가 같은 필수 field를 반환한다.
- 경계 — provider 실패 시 허용 여부에 따라 fallback 또는 구조화 오류가 난다.
- 외부 쓰기 — 기본값 false · dry-run과 사람 승인 뒤에만 별도 adapter 호출
- 기록 — status·error_code·입력 식별자·meeting_brief.json

어떤 파일을 열고 어디에서 결과를 확인하는지 먼저 봅시다

입력·fixture

```
src/langchain_lab.py
```

구현 코드

```
src/openai_provider.py
```

검증·test

```
requirements-local-llm-optional.txt
```

따라하기 notebook

```
tests/test_langchain_langgraph_lab.py
```

LangChain은 prompt·model·parser를 하나의 실행 계약으로 묶습니다

provider별 코드를 업무 로직에 섞으면 모델 변경이 곧 전체 재개발이 된다.

provider_requested·provider_used·fallback_reason을 결과에 함께 남긴다.

이 차시에서
버릴 오해

OpenAI 오류가 조용히
fixture 성공처럼 보인다.

비슷해 보이는 선택도 책임과 검증 범위가 다릅니다

구분	역할	이번 차시의 판단 기준
Runnable	같은 invoke 계약으로 실행되는 구성요소	결정론 test로 먼저 확인
LCEL	Runnable을 연결하는 표현 방식	adapter 경계를 확인
Adapter	provider 차이를 흡수하는 층	비용·지연·권한을 확인
Fallback	실패 시 정해진 대체 경로	사람 판단과 운영 기록을 확인

가장 먼저 재현할 실패는 이것입니다

실패

OpenAI 오류가 조용히 fixture
성공처럼 보인다.

사용자 영향

provider별 코드를 업무 로직에 섞으면 모델
변경이 곧 전체 재개발이 된다.

실패가 재현돼야 복구 코드와 회귀 test를 만들 수 있습니다.

복구는 재시도보다 먼저 중단 조건을 정합니다

- 01 — 탐지: provider 실패 시 허용 여부에 따라 fallback 또는 구조화 오류가 난다.
- 02 — 중단: OpenAI 오류가 조용히 fixture 성공처럼 보인다.
- 03 — 복구: provider_requested·provider_used·fallback_reason을 결과에 함께 남긴다.
- 04 — 증거: meeting_brief.json과 test 결과를 함께 남긴다.

Codex는 코드를 대신 쓰는 도구보다 검증 루프에 가깝습니다

새 provider adapter가 기존 MeetingBrief contract와 fallback test를 통과하게 한다.

저장소 이해

→ 허용 범위 변경

→ focused test

→ diff 리뷰

→ 사람 merge

6차시 · LangChain LCEL Provider 교체

같은 transcript와 Pydantic schema를 유지하고 provider만 바꿉니다.

Prompt → fixture/Ollama → Pydantic Parser → Policy Validator

Ollama용 선택 패키지는 다음과 같이 설치합니다.

python -m pip install -r requirements-local-llm-optional.txt

```
In [10]: from src.langchain_lab import run_langchain_lab

transcript = (ROOT / "data/meeting_sample_ko_12min.txt").read_text(encoding="utf-8")
fixture_chain = run_langchain_lab(transcript, provider="fixture")
print(json.dumps({
    "provider_used": fixture_chain["provider_used"],
    "pipeline": fixture_chain["pipeline"],
    "checks": fixture_chain["checks"],
}, ensure_ascii=False, indent=2))

{
  "provider_used": "fixture",
  "pipeline": [
    "ChatPromptTemplate",
    "Model Adapter",
    "PydanticOutputParser",
    "Policy Validator"
  ],
  "checks": {
    "schema_valid": true,
    "evidence_present": true,
    "automatic_email_blocked": true,
    "human_approval_required": true
  }
}

In [11]: ollama_chain = run_langchain_lab(
    transcript,
    provider="ollama",
    model=MODEL,
    allow_fallback=True,
```

Codex에게는 목표보다 완료 조건을 더 구체적으로 줍니다

목표

새 provider adapter가 기존 MeetingBrief contract와 fallback test를 통과하게 한다.

허용 경로

src/openai_provider.py
requirements-local-llm-optional.txt

완료 조건

Qwen과 fixture가 같은 필수 field를 반환한다.
provider 실패 시 허용 여부에 따라 fallback 또는 구조화 오류가 난다.

금지

secret 출력 · workspace 밖 변경 · test 약화 · 사람 승인 없는 외부 쓰기

생성된 patch는 diff와 test 증거를 보고 사람이 결정합니다

- 01 **Scope** — 허용 경로 밖 파일이 바뀌지 않았는가
- 02 **Focused test** — Qwen과 fixture가 같은 필수 field를 반환한다.
- 03 **Boundary test** — provider 실패 시 허용 여부에 따라 fallback 또는 구조화 오류가 난다.
- 04 **Human merge** — Codex 리뷰와 test 통과는 자동 승인이 아니다

강사가 먼저 정상 경로를 끝까지 실행합니다

01 — 열기: `src/langchain_lab.py`

02 — 구현 확인: `src/openai_provider.py`

03 — 실행: `python -m src.langchain_lab --provider ollama --model qwen3:4b`

04 — 성공 신호: Qwen과 fixture가 같은 필수 field를 반환한다.

같은 저장소에서 이 명령을 실행합니다

```
$ python -m src.langchain_lab --provider ollama --model qwen3:4b
```

실행 전 확인

현재 경로가 repository root인지 · Python 3.12 환경인지 · .env가 출력되지 않는지

완료 여부는 화면이 아니라 결과 계약으로 확인합니다

STATUS

SUCCESS 또는 EXPECTED_FAILURE

ARTIFACT

meeting_brief.json

사람이 확인할 세 줄

1. Qwen과 fixture가 같은 필수 field를 반환한다.
2. provider 실패 시 허용 여부에 따라 fallback 또는 구조화 오류가 난다.
3. external_write=false

강사는 실패 한 건도 바로 재현하고 복구합니다

재현

OpenAI 오류가 조용히 fixture 성공처럼 보인다.

복구

provider_requested·provider_used·fallback_reason을 결과에 함께 남긴다.

확인: provider 실패 시 허용 여부에 따라 fallback 또는 구조화 오류가 난다.

이제 같은 코드를 각자 실행하고 한 줄만 바꿉니다

열 파일

tests/test_langchain_langgraph_lab.py

수정 범위

notebook의 실습 변수 또는 fixture 한 줄 · src 전체를 복사해 바꾸지 않음

완료 기준

Qwen과 fixture가 같은 필수 field를 반환한다.
meeting_brief.json 저장

STEP 1 · 입력과 설정을 확인합니다

파일: `src/langchain_lab.py`
변수: 실행 경로·provider·dry-run 여부

결과 파일: `meeting_brief.json`

STEP 2 · 정상 경로를 실행합니다

```
python -m src.langchain_lab --provider ollama --model qwen3:4b
```

성공 신호: Qwen과 fixture가 같은 필수 field를 반환한다.

결과 파일: meeting_brief.json

STEP 3 · 경계값을 바꿔 실패를 확인합니다

provider 실패 시 허용 여부에 따라 fallback 또는 구조화 오류가 난다.

복구: provider_requested·provider_used·fallback_reason을 결과에 함께 남긴다.

결과에 error_code와 external_write=false가 보이면 완료

정상 case를 focused test로 확인합니다

NORMAL

Qwen과 fixture가 같은 필수 field를 반환한다.

- 01 — 입력 fixture와 실행 command가 기록됐는가
- 02 — 결과 schema가 validate되는가
- 03 — focused test가 실제로 통과했는가

가장 중요한 실패 조건을 test로 고정합니다

BOUNDARY

provider 실패 시 허용 여부에 따라 fallback 또는 구조화 오류가 난다.

- 01 — raw traceback 대신 stable error_code가 남는가
- 02 — 외부 쓰기·자동 메일이 false인가
- 03 — 실패가 다음 차시에서 재현 가능한가

재직자는 작은 운영 문제 한 곳에 먼저 적용합니다

비용·보안·지연에 따라 로컬과 API 모델을 교체하는 회의 서비스

- 01 — 실제 업무 데이터 대신 합성·비식별 sample로 먼저 실행
- 02 — 현재 수작업 시간과 오류를 baseline으로 기록
- 03 — 사람 승인 전에는 외부 시스템을 바꾸지 않음
- 04 — meeting_brief.json을 운영 검토 자료로 사용

구직자는 제품 판단과 검증 증거를 함께 보여줍니다

비용·보안·지연에 따라 로컬과 API 모델을 교체하는 회의 서비스

- 01 — 문제와 사용자 영향을 한 문장으로 설명
- 02 — 정상 demo와 대표 실패 demo를 함께 준비
- 03 — Codex가 만든 diff와 직접 검토한 test 증거를 구분
- 04 — meeting_brief.json과 README 재현 절차를 제출

서비스로 운영하려면 이 네 가지 질문에 답해야 합니다

품질

무엇을 Qwen과 fixture가 같은 필수 field를 반환한다.로 정의하는가

복구

provider_requested·provider_used·fallback_reason을 결과에 함께 남긴다.

안전

어디에서 사람 승인과 external_write=false를 확인하는가

관측

meeting_brief.json에서 어떤 status·error_code를 보는가

6차시를 마치기 전에 세 가지를 확인합니다

- 01 — 설명: LangChain은 prompt·model·parser를 하나의 실행 계약으로 묶습니다이 필요한 이유를 한 문장으로 말할 수 있다.
- 02 — 실행: `python -m src.langchain_lab --provider ollama --model qwen3:4b`
- 03 — 증거: meeting_brief.json과 정상·실패 test 결과가 남아 있다.

다음 차시: 근거가 없는 Action Item은 좋은 문장이 아니라 위험한 업무입니다

15:30-16:20 (쉬는 시간 17:10-17:40)

근거가 없는 Action Item은 좋은 문장이 아니라 위험한 업무입니다

모든 결정과 할 일을 원문 segment ID에 연결한다.

진행

강의 12분 · 강사 시연 10분 · 소프트웨어
실습 23분 · 실행 확인 5분

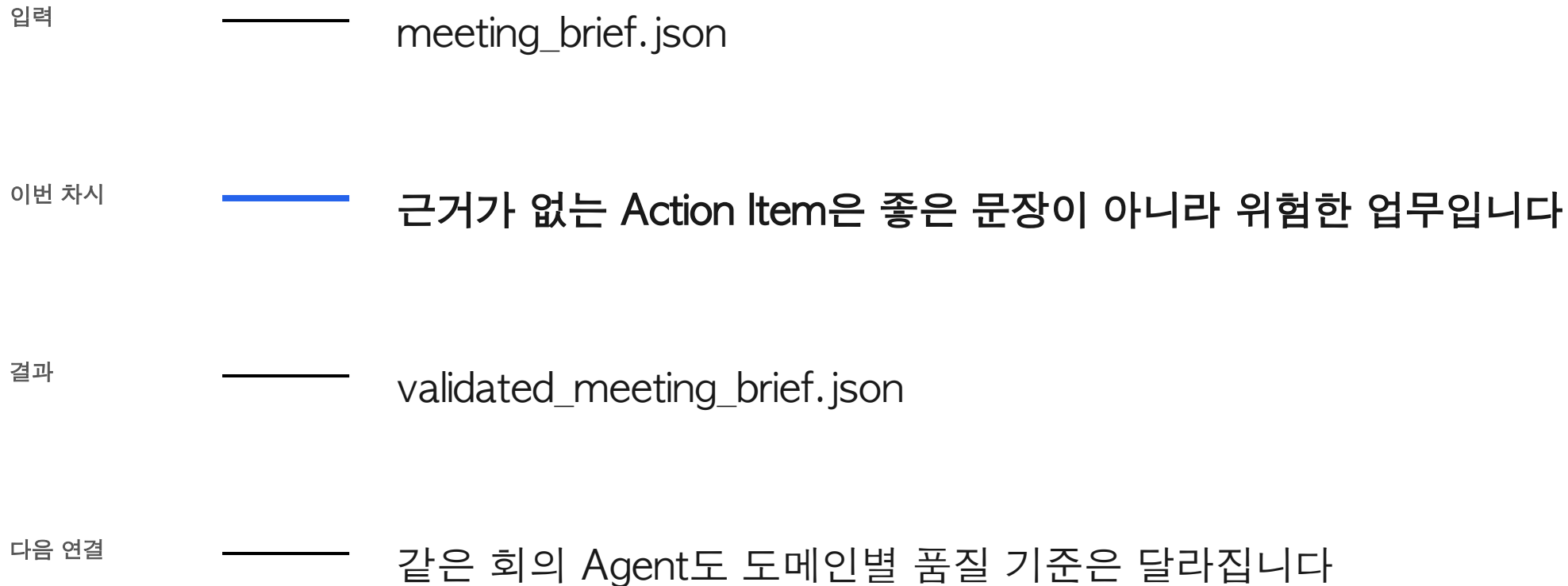
사용 파일

src/meeting_agent_workflow.py
src/course_services/meeting_service.py

확인할 결과

validated_meeting_brief.json

7차시는 앞의 결과를 이어받아 다음 상태를 만듭니다



담당자와 기한을 잘못 붙이면 회의 자동화가 오히려 조정 비용을 만든다.

모든 결정과 할 일을 원문 segment ID에 연결한다.

이번 차시의 확인 결과: `validated_meeting_brief.json`

이 차시가 끝나면 세 가지를 설명하고 실행할 수 있습니다

- 01 — 모든 결정과 할 일을 원문 segment ID에 연결한다.
- 02 — evidence ID가 존재하지만 실제 문장과 의미가 맞지 않는다. 왜 위험한지 설명한다.
- 03 — validated_meeting_brief.json에서 정상과 실패 상태를 직접 확인한다.

처음 보는 용어는 이 네 가지 역할만 구분합니다

용어	이 수업에서 쓰는 뜻
Evidence	결과를 뒷받침하는 원문 구간
Grounding	출력 내용을 입력 근거에 묶는 것
Hallucination	원문에 없는 내용을 만든 오류
Spot check	위험 구간만 사람이 확인

입력·처리·결과·검증을 분리하면 문제가 빨리 보입니다

INPUT

src/meeting_agent_workflow.py

PROCESS

결정 추출 → Action Item 추출 → evidence ID 연결

OUTPUT

validated_meeting_brief.json

VERIFY

모든 evidence ID가 known segment에 존재한다.
없는 s99를 가리키면
ACTION_n_UNKNOWN_EVIDENCE다.

실행 흐름은 다섯 단계로 고정합니다



마지막 판단은 validated_meeting_brief.json에서 사람이 확인합니다.

코드보다 먼저 성공·실패 계약을 정합니다

정상	——	모든 evidence ID가 known segment에 존재한다.
경계	——	없는 s99를 가리키면 ACTION_n_UNKNOWN_EVIDENCE다.
외부 쓰기	——	기본값 false · dry-run과 사람 승인 뒤에만 별도 adapter 호출
기록	——	status·error_code·입력 식별자·validated_meeting_brief.json

어떤 파일을 열고 어디에서 결과를 확인하는지 먼저 봅시다

입력·fixture

```
src/meeting_agent_workflow.py
```

구현 코드

```
src/course_services/meeting_service.py
```

검증·test

```
tests/test_meeting_agent_workflow.py
```

따라하기 notebook

```
output/day1-meeting-agent-ollama/workflow_result.json
```

근거가 없는 Action Item은 좋은 문장이 아니라 위험한 업무입니다

담당자와 기한을 잘못 붙이면 회의 자동화가 오히려 조정 비용을 만든다.

구조 검사는 code evaluator, 의미 검사는 표본 human review로 나눈다.

이 차시에서
버릴 오해

evidence ID가
존재하지만 실제 문장과
의미가 맞지 않는다.

비슷해 보이는 선택도 책임과 검증 범위가 다릅니다

구분	역할	이번 차시의 판단 기준
Evidence	결과를 뒷받침하는 원문 구간	결정론 test로 먼저 확인
Grounding	출력 내용을 입력 근거에 묶는 것	adapter 경계를 확인
Hallucination	원문에 없는 내용을 만든 오류	비용·지연·권한을 확인
Spot check	위험 구간만 사람이 확인	사람 판단과 운영 기록을 확인

가장 먼저 재현할 실패는 이것입니다

실패

evidence ID가 존재하지만 실제 문장과 의미가 맞지 않는다.

사용자 영향

담당자와 기한을 잘못 붙이면 회의 자동화가 오히려 조정 비용을 만든다.

실패가 재현돼야 복구 코드와 회귀 test를 만들 수 있습니다.

복구는 재시도보다 먼저 중단 조건을 정합니다

- 01 — 탐지: 없는 s99를 가리키면 ACTION_n_UNKNOWN_EVIDENCE다.
- 02 — 중단: evidence ID가 존재하지만 실제 문장과 의미가 맞지 않는다.
- 03 — 복구: 구조 검사는 code evaluator, 의미 검사는 표본 human review로 나눈다.
- 04 — 증거: validated_meeting_brief.json과 test 결과를 함께 남긴다.

Codex는 코드를 대신 쓰는 도구보다 검증 루프에 가깝습니다

Action Item evidence 누락
·unknown ID를 각각 재현하는
test를 추가한다.

저장소 이해
→ 허용 범위 변경
→ focused test
→ diff 리뷰
→ 사람 merge

WORKFLOW LAB

Pipeline Review Result

02 LANGGRAPH INTERRUPT

결과를 내보내기 전에 사람이 결정합니다.

REVIEW_REQUIRED

고객 문의 자동화 PoC 범위 회의

배송 지연과 반품 절차 안내만 1차 자동화 범위로 정하고, 외부 발행 전 사람이 검토한다.

ACTION ITEMS

공개 FAQ 30건 비식별 샘플 정리

서연 · 2026-08-27

근거 s12, s18

응답 JSON Schema와 실패 테스트 작성

준호 · 2026-08-28

근거 s27, s31

interrupt_payload.json

read only

```
{
  "question": "회의 결과를 승인·수정·거절하시겠습니까?",
  "request_id": "meeting-approve-001",
  "summary": "배송 지연과 반품 절차 안내만 1차 자동화 범위로 정하고, 외부 발행 전 사람이 검토한다.",
  "action_items": [
    {
      "task": "공개 FAQ 30건 비식별 샘플 정리",
      "owner": "서연",
      "due_date": "2026-08-27",
      "evidence_ids": [
        "s12",
        "s18"
      ]
    },
    {
      "task": "응답 JSON Schema와 실패 테스트 작성",
      "owner": "준호"
    }
  ]
}
```

Codex에게는 목표보다 완료 조건을 더 구체적으로 줍니다

목표

Action Item evidence 누락·unknown ID를 각각 재현하는 test를 추가한다.

허용 경로

src/course_services/meeting_service.py
tests/test_meeting_agent_workflow.py

완료 조건

모든 evidence ID가 known segment에 존재한다.
없는 s99를 가리키면 ACTION_n_UNKNOWN_EVIDENCE다.

금지

secret 출력 · workspace 밖 변경 · test 약화 · 사람 승인 없는 외부 쓰기

생성된 patch는 diff와 test 증거를 보고 사람이 결정합니다

- 01 **Scope** — 허용 경로 밖 파일이 바뀌지 않았는가
- 02 **Focused test** — 모든 evidence ID가 known segment에 존재한다.
- 03 **Boundary test** — 없는 s99를 가리키면 ACTION_n_UNKNOWN_EVIDENCE다.
- 04 **Human merge** — Codex 리뷰와 test 통과는 자동 승인이 아니다

강사가 먼저 정상 경로를 끝까지 실행합니다

- 01 — 열기: `src/meeting_agent_workflow.py`
- 02 — 구현 확인: `src/course_services/meeting_service.py`
- 03 — 실행: `python -m src.meeting_agent_workflow --help`
- 04 — 성공 신호: 모든 evidence ID가 known segment에 존재한다.

같은 저장소에서 이 명령을 실행합니다

```
$ python -m src.meeting_agent_workflow --help
```

실행 전 확인

현재 경로가 repository root인지 · Python 3.12 환경인지 · .env가 출력되지 않는지

완료 여부는 화면이 아니라 결과 계약으로 확인합니다

STATUS

SUCCESS 또는 EXPECTED_FAILURE

ARTIFACT

validated_meeting_brief.json

사람이 확인할 세 줄

1. 모든 evidence ID가 known segment에 존재한다.
2. 없는 s99를 가리키면 ACTION_n_UNKNOWN_EVIDENCE다.
3. external_write=false

강사는 실패 한 건도 바로 재현하고 복구합니다

재현

evidence ID가 존재하지만 실제 문장과 의미가 맞지 않는다.

복구

구조 검사는 code evaluator, 의미 검사는 표본 human review로 나눈다.

확인: 없는 s99를 가리키면 ACTION_n_UNKNOWN_EVIDENCE다.

이제 같은 코드를 각자 실행하고 한 줄만 바꿉니다

열 파일

output/day1-meeting-agent-ollama/workflow_result.json

수정 범위

notebook의 실습 변수 또는 fixture 한 줄 · src 전체를 복사해 바꾸지 않음

완료 기준

모든 evidence ID가 known segment에 존재한다.
validated_meeting_brief.json 저장

STEP 1 · 입력과 설정을 확인합니다

파일: `src/meeting_agent_workflow.py`
변수: 실행 경로·`provider·dry-run` 여부

결과 파일: `validated_meeting_brief.json`

STEP 2 · 정상 경로를 실행합니다

```
python -m src.meeting_agent_workflow --help
```

성공 신호: 모든 evidence ID가 known segment에 존재한다.

결과 파일: validated_meeting_brief.json

STEP 3 · 경계값을 바꿔 실패를 확인합니다

없는 s99를 가리키면 ACTION_n_UNKNOWN_EVIDENCE다.

복구: 구조 검사는 code evaluator, 의미 검사는 표본 human review로 나눈다.

결과에 error_code와 external_write=false가 보이면 완료

정상 case를 focused test로 확인합니다

NORMAL

모든 evidence ID가 known segment에 존재한다.

- 01 — 입력 fixture와 실행 command가 기록됐는가
- 02 — 결과 schema가 validate되는가
- 03 — focused test가 실제로 통과했는가

가장 중요한 실패 조건을 test로 고정합니다

BOUNDARY

없는 s99를 가리키면 ACTION_n_UNKNOWN_EVIDENCE다.

- 01 — raw traceback 대신 stable error_code가 남는가
- 02 — 외부 쓰기·자동 메일이 false인가
- 03 — 실패가 다음 차시에서 재현 가능한가

재직자는 작은 운영 문제 한 곳에 먼저 적용합니다

회의 후 담당 업무를 자동 제안하되 게시 전 원문 근거를 확인

- 01 — 실제 업무 데이터 대신 합성·비식별 sample로 먼저 실행
- 02 — 현재 수작업 시간과 오류를 baseline으로 기록
- 03 — 사람 승인 전에는 외부 시스템을 바꾸지 않음
- 04 — validated_meeting_brief.json을 운영 검토 자료로 사용

구직자는 제품 판단과 검증 증거를 함께 보여줍니다

회의 후 담당 업무를 자동 제안하되 게시 전 원문 근거를 확인

- 01 — 문제와 사용자 영향을 한 문장으로 설명
- 02 — 정상 demo와 대표 실패 demo를 함께 준비
- 03 — Codex가 만든 diff와 직접 검토한 test 증거를 구분
- 04 — validated_meeting_brief.json과 README 재현 절차를 제출

서비스로 운영하려면 이 네 가지 질문에 답해야 합니다

품질

무엇을 모든 evidence ID가 known segment에 존재한다.로 정의하는가

안전

어디에서 사람 승인과 external_write=false를 확인하는가

복구

구조 검사는 code evaluator, 의미 검사는 표본 human review로 나눈다.

관측

validated_meeting_brief.json에서 어떤 status:error_code를 보는가

7차시를 마치기 전에 세 가지를 확인합니다

- 01 — 설명: 근거가 없는 Action Item은 좋은 문장이 아니라 위험한 업무입니다이 필요한 이유를 한 문장으로 말할 수 있다.
- 02 — 실행: `python -m src.meeting_agent_workflow --help`
- 03 — 증거: `validated_meeting_brief.json`과 정상·실패 test 결과가 남아 있다.

다음 차시: 같은 회의 Agent도 도메인별 품질 기준은 달라집니다

16:20-17:10 (쉬는 시간 17:10-17:40)

같은 회의 Agent도 도메인별 품질 기준은 달라집니다

이커머스·교육·금융 사례에 맞는 평가 case를 만든다.

진행

강의 12분 · 강사 시연 10분 · 소프트웨어
실습 23분 · 실행 확인 5분

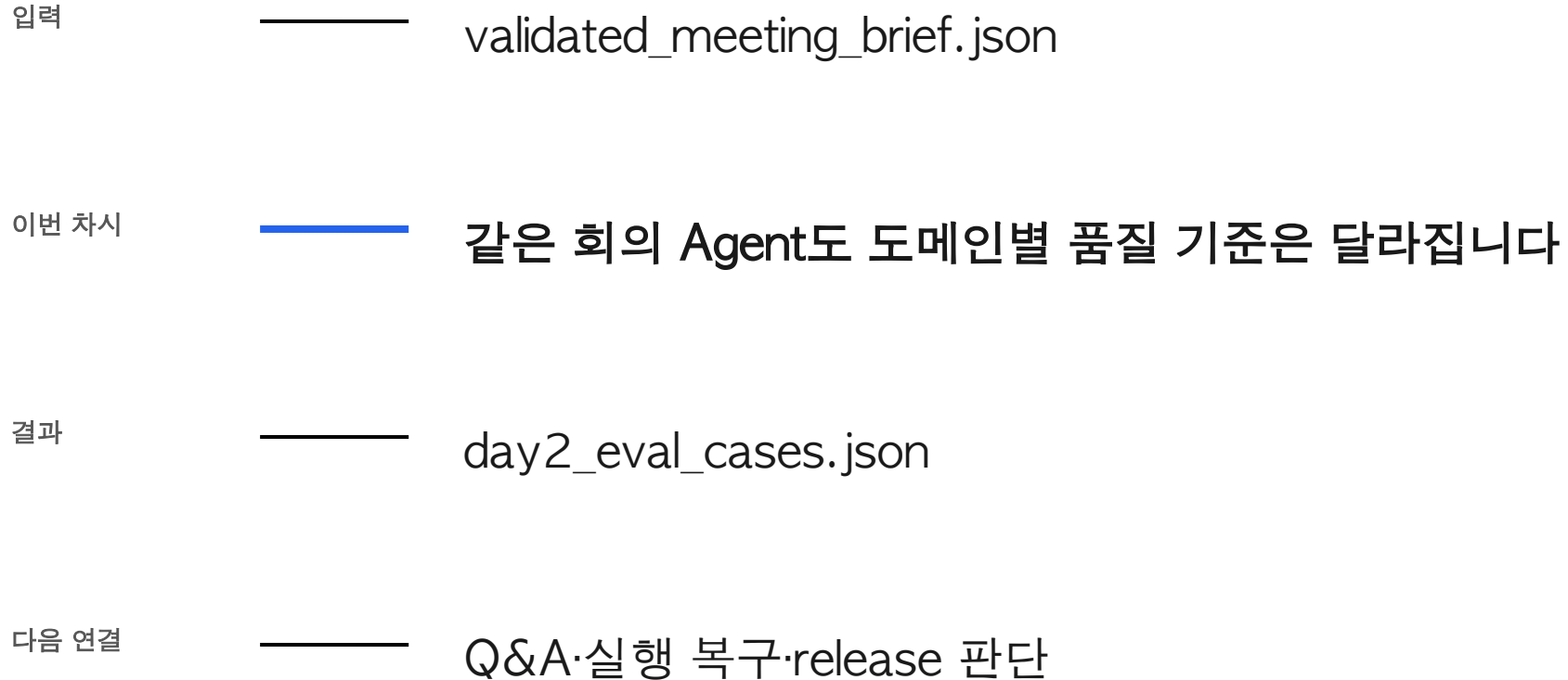
사용 파일

data/
meeting_sample_ko_12min_expected.json
src/observability_lab.py

확인할 결과

day2_eval_cases.json

8차시는 앞의 결과를 이어받아 다음 상태를 만듭니다



범용 요약 점수만으로는 금액·날짜·상품명 같은 도메인 오류를 잡지 못한다.

이커머스·교육·금융 사례에 맞는 평가 case를 만든다.

이번 차시의 확인 결과: day2_eval_cases.json

이 차시가 끝나면 세 가지를 설명하고 실행할 수 있습니다

- 01 — 이커머스·교육·금융 사례에 맞는 평가 case를 만든다.
- 02 — 데모 한 건 성공을 서비스 전체 품질로 일반화한다. 왜 위험한지 설명한다.
- 03 — day2_eval_cases.json에서 정상과 실패 상태를 직접 확인한다.

처음 보는 용어는 이 네 가지 역할만 구분합니다

용어	이 수업에서 쓰는 뜻
Golden case	팀이 합의한 대표 입력과 기대 결과
Evaluator	결과를 검사하는 함수
Regression	변경 후 이전 동작이 깨진 상태
Release gate	배포 여부를 가르는 기준

입력·처리·결과·검증을 분리하면 문제가 빨리 보입니다

INPUT

data/meeting_sample_ko_12min_expected.json

PROCESS

실패 사례 수집 → 비식별 → expected field 작성

OUTPUT

day2_eval_cases.json

VERIFY

합성 회의 정상 case가 READY_FOR_EXPORT다.
STT HOLD나 사람 거절은 최종 READY가 될 수 없다.

실행 흐름은 다섯 단계로 고정합니다

01

실패 사례 수집

02

비식별

03

expected field 작성

04

baseline 실행

05

Day 2 tag

마지막 판단은 `day2_eval_cases.json`에서 사람이 확인합니다.

코드보다 먼저 성공·실패 계약을 정합니다

- 정상 ——— 합성 회의 정상 case가 READY_FOR_EXPORT다.
- 경계 ——— STT HOLD나 사람 거절은 최종 READY가 될 수 없다.
- 외부 쓰기 ——— 기본값 false · dry-run과 사람 승인 뒤에만 별도 adapter 호출
- 기록 ——— status·error_code·입력 식별자·day2_eval_cases.json

어떤 파일을 열고 어디에서 결과를 확인하는지 먼저 봅시다

입력·fixture

```
data/meeting_sample_ko_12min_expected.json
```

구현 코드

```
src/observability_lab.py
```

검증·test

```
tests/test_observability_workflow.py
```

따라하기 notebook

```
materials/day2/day2_service_lab.ipynb
```

같은 회의 Agent도 도메인별 품질 기준은 달라집니다

범용 요약 점수만으로는 금액·날짜·상품명 같은 도메인 오류를 잡지 못한다.

숫자·이름·기한·무음·반복을 포함한 작은 golden set부터 운영한다.

이 차시에서
버릴 오해

데모 한 건 성공을
서비스 전체 품질로
일반화한다.

비슷해 보이는 선택도 책임과 검증 범위가 다릅니다

구분	역할	이번 차시의 판단 기준
Golden case	팀이 합의한 대표 입력과 기대 결과	결정론 test로 먼저 확인
Evaluator	결과를 검사하는 함수	adapter 경계를 확인
Regression	변경 후 이전 동작이 깨진 상태	비용·지연·권한을 확인
Release gate	배포 여부를 가르는 기준	사람 판단과 운영 기록을 확인

가장 먼저 재현할 실패는 이것입니다

실패

데모 한 건 성공을 서비스 전체
품질로 일반화한다.

사용자 영향

범용 요약 점수만으로는 금액·날짜·상품명
같은 도메인 오류를 잡지 못한다.

실패가 재현돼야 복구 코드와 회귀 test를 만들 수 있습니다.

복구는 재시도보다 먼저 중단 조건을 정합니다

- 01 — 탐지: STT HOLD나 사람 거절은 최종 READY가 될 수 없다.
- 02 — 중단: 데모 한 건 성공을 서비스 전체 품질로 일반화한다.
- 03 — 복구: 숫자·이름·기한·무음·반복을 포함한 작은 golden set부터 운영한다.
- 04 — 증거: day2_eval_cases.json과 test 결과를 함께 남긴다.

Codex는 코드를 대신 쓰는 도구보다 검증 루프에 가깝습니다

대표 실패를 재현하는 test를
먼저 쓰고 최소 변경으로
통과시킨다.

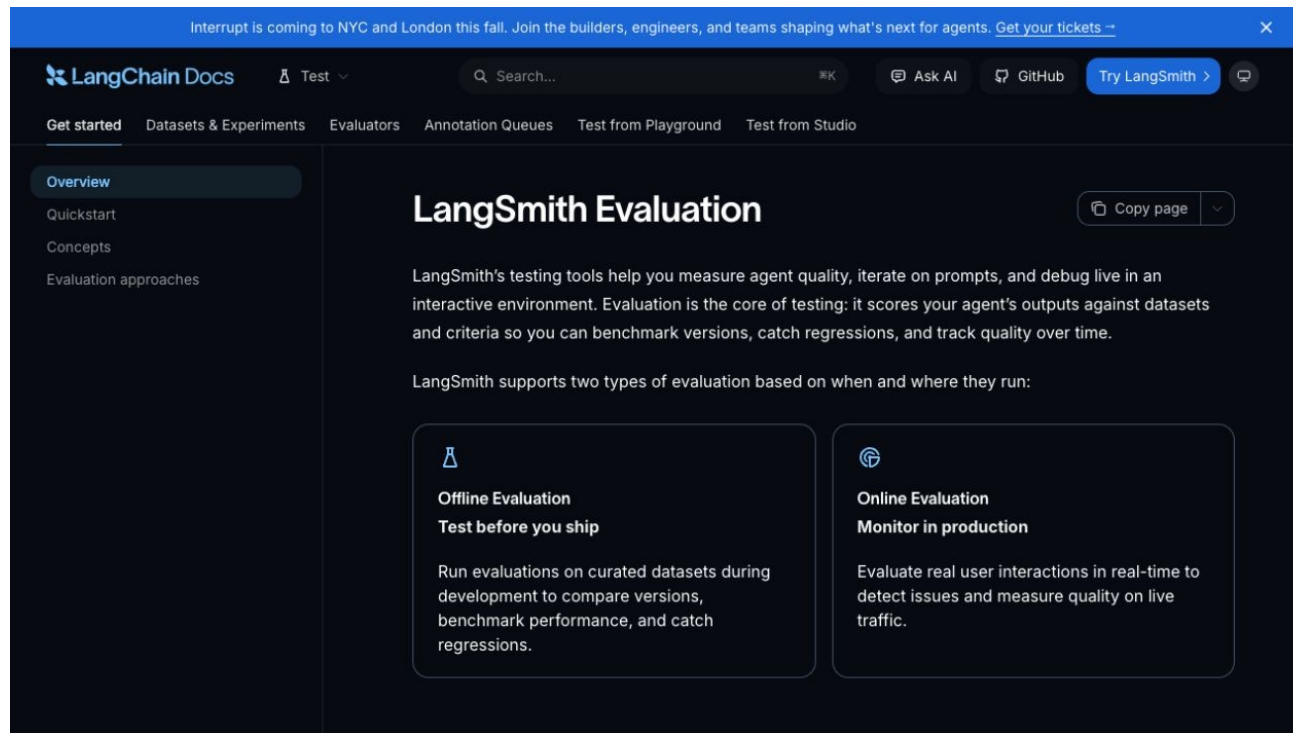
저장소 이해

→ 허용 범위 변경

→ focused test

→ diff 리뷰

→ 사람 merge



Codex에게는 목표보다 완료 조건을 더 구체적으로 줍니다

목표

대표 실패를 재현하는 test를 먼저 쓰고 최소 변경으로 통과시킨다.

허용 경로

src/observability_lab.py
tests/test_observability_workflow.py

완료 조건

합성 회의 정상 case가 READY_FOR_EXPORT다.
STT HOLD나 사람 거절은 최종 READY가 될 수 없다.

금지

secret 출력 · workspace 밖 변경 · test 약화 · 사람 승인 없는 외부 쓰기

생성된 patch는 diff와 test 증거를 보고 사람이 결정합니다

- 01 **Scope** — 허용 경로 밖 파일이 바뀌지 않았는가
- 02 **Focused test** — 합성 회의 정상 case가 READY_FOR_EXPORT다.
- 03 **Boundary test** — STT HOLD나 사람 거절은 최종 READY가 될 수 없다.
- 04 **Human merge** — Codex 리뷰와 test 통과는 자동 승인이 아니다

강사가 먼저 정상 경로를 끝까지 실행합니다

- 01 — 열기: `data/meeting_sample_ko_12min_expected.json`
- 02 — 구현 확인: `src/observability_lab.py`
- 03 — 실행: `python -m pytest -q tests/test_meeting_agent_workflow.py tests/test_observability_workflow.py`
- 04 — 성공 신호: 합성 회의 정상 case가 `READY_FOR_EXPORT`다.

같은 저장소에서 이 명령을 실행합니다

```
$ python -m pytest -q tests/test_meeting_agent_workflow.py  
tests/test_observability_workflow.py
```

실행 전 확인

현재 경로가 repository root인지 · Python 3.12 환경인지 · .env가 출력되지 않는지

완료 여부는 화면이 아니라 결과 계약으로 확인합니다

STATUS

SUCCESS 또는 EXPECTED_FAILURE

ARTIFACT

day2_eval_cases.json

사람이 확인할 세 줄

1. 합성 회의 정상 case가 READY_FOR_EXPORT다.
2. STT HOLD나 사람 거절은 최종 READY가 될 수 없다.
3. external_write=false

강사는 실패 한 건도 바로 재현하고 복구합니다

재현

데모 한 건 성공을 서비스 전체 품질로 일반화한다.

복구

숫자·이름·기한·무음·반복을 포함한 작은 golden set부터 운영한다.

확인: STT HOLD나 사람 거절은 최종 READY가 될 수 없다.

이제 같은 코드를 각자 실행하고 한 줄만 바꿉니다

열 파일

materials/day2/day2_service_lab.ipynb

수정 범위

notebook의 실습 변수 또는 fixture 한 줄 · src 전체를 복사해 바꾸지 않음

완료 기준

합성 회의 정상 case가 READY_FOR_EXPORT다.
day2_eval_cases.json 저장

STEP 1 · 입력과 설정을 확인합니다

파일: data/meeting_sample_ko_12min_expected.json
변수: 실행 경로·provider·dry-run 여부

결과 파일: day2_eval_cases.json

STEP 2 · 정상 경로를 실행합니다

```
python -m pytest -q tests/test_meeting_agent_workflow.py  
tests/test_observability_workflow.py
```

성공 신호: 합성 회의 정상 case가 READY_FOR_EXPORT다.

결과 파일: day2_eval_cases.json

STEP 3 · 경계값을 바꿔 실패를 확인합니다

STT HOLD나 사람 거절은 최종 READY가 될 수 없다.
복구: 숫자·이름·기한·무음·반복을 포함한 작은 golden set부터 운영한다.

결과에 `error_code`와 `external_write=false`가 보이면 완료

정상 case를 focused test로 확인합니다

NORMAL

합성 회의 정상 case가 READY_FOR_EXPORT다.

- 01 — 입력 fixture와 실행 command가 기록됐는가
- 02 — 결과 schema가 validate되는가
- 03 — focused test가 실제로 통과했는가

가장 중요한 실패 조건을 test로 고정합니다

BOUNDARY

STT HOLD나 사람 거절은 최종 READY가 될 수 없다.

- 01 — raw traceback 대신 stable error_code가 남는가
- 02 — 외부 쓰기·자동 메일이 false인가
- 03 — 실패가 다음 차시에서 재현 가능한가

재직자는 작은 운영 문제 한 곳에 먼저 적용합니다

현업에서는 상담/회의 유형별 scorecard, 포트폴리오에는 실패 사례 3건

- 01 — 실제 업무 데이터 대신 합성·비식별 sample로 먼저 실행
- 02 — 현재 수작업 시간과 오류를 baseline으로 기록
- 03 — 사람 승인 전에는 외부 시스템을 바꾸지 않음
- 04 — day2_eval_cases.json을 운영 검토 자료로 사용

구직자는 제품 판단과 검증 증거를 함께 보여줍니다

현업에서는 상담/회의 유형별 scorecard, 포트폴리오에는 실패 사례 3건

- 01 — 문제와 사용자 영향을 한 문장으로 설명
- 02 — 정상 demo와 대표 실패 demo를 함께 준비
- 03 — Codex가 만든 diff와 직접 검토한 test 증거를 구분
- 04 — day2_eval_cases.json과 README 재현 절차를 제출

서비스로 운영하려면 이 네 가지 질문에 답해야 합니다

품질

무엇을 합성 회의 정상 case가
READY_FOR_EXPORT다.로 정의하는가

안전

어디에서 사람 승인과 external_write=false를
확인하는가

복구

숫자·이름·기한·무음·반복을 포함한 작은 golden set
부터 운영한다.

관측

day2_eval_cases.json에서 어떤 status·error_code
를 보는가

8차시를 마치기 전에 세 가지를 확인합니다

- 01 — 설명: 같은 회의 Agent도 도메인별 품질 기준은 달라집니다이 필요한 이유를 한 문장으로 말할 수 있다.
- 02 — 실행: `python -m pytest -q tests/test_meeting_agent_workflow.py tests/test_observability_workflow.py`
- 03 — 증거: `day2_eval_cases.json`과 정상·실패 test 결과가 남아 있다.

17:10-17:40 쉬는 시간 · 17:40-18:00 Q&A·실행 복구